МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра №806 «Вычислительная математика и программирование»

Отчёт

по предмету «Численные методы»

Выполнил: Ситников И. А.
Группа: М8О-307Б-23
Преподаватель: Киндинова В. В.
Вариант: 24

Москва 2026

Содержание

1 Решение СЛАУ **4**
1.1 Задание 1.1 . . . 4
1.1.1 Формулировка задания . . . 4
1.1.2 Вариант № 24 . . . 4
1.1.3 Основные формулы . . . 4
1.1.4 Текст программы . . . 4
1.2 Задание 1.2 . . . 8
1.2.1 Формулировка задания . . . 8
1.2.2 Вариант № 24 . . . 8
1.2.3 Основные формулы . . . 8
1.2.4 Текст программы . . . 9
1.3 Задание 1.3 . . . 11
1.3.1 Формулировка задания . . . 11
1.3.2 Вариант № 24 . . . 11
1.3.3 Основные формулы . . . 11
1.3.4 Текст программы . . . 11
1.3.5 Анализ результатов . . . 16
1.4 Задание 1.4 . . . 16
1.4.1 Формулировка задания . . . 16
1.4.2 Вариант № 24 . . . 16
1.4.3 Основные формулы . . . 16
1.4.4 Текст программы . . . 17
1.4.5 Анализ результатов . . . 21
1.5 Задание 1.5 . . . 21
1.5.1 Формулировка задания . . . 21
1.5.2 Вариант № 24 . . . 21
1.5.3 Основные формулы . . . 21
1.5.4 Текст программы . . . 22
1.5.5 Анализ результатов . . . 29
1.6 Выводы . . . 29

2 Решение нелинейных уравнений **29**
2.1 Задание 2.1 . . . 29
2.1.1 Формулировка задания . . . 29
2.1.2 Вариант № 24 . . . 29
2.1.3 Основные формулы . . . 30
2.1.4 Текст программы . . . 30
2.1.5 Анализ сходимости . . . 39
2.2 Задание 2.2 . . . 39
2.2.1 Формулировка задания . . . 39
2.2.2 Вариант № 24 . . . 39
2.2.3 Основные формулы . . . 39
2.2.4 Графическое определение начального приближения . . . 39
2.2.5 Текст программы . . . 40
2.2.6 Анализ сходимости . . . 48
2.3 Выводы . . . 49

3 Интерполяция и аппроксимация **49**
3.1 Задание 3.1 . . . 49

3.1.1 Формулировка задания . . . 49
3.1.2 Вариант № 24 . . . 49
3.1.3 Основные формулы . . . 49
3.1.4 Текст программы . . . 50
3.1.5 Анализ результатов . . . 57
3.2 Задание 3.2 . . . 57
3.2.1 Формулировка задания . . . 57
3.2.2 Вариант № 24 . . . 58
3.2.3 Основные формулы . . . 58
3.2.4 Текст программы . . . 58
3.2.5 Анализ результатов . . . 63
3.3 Задание 3.3 . . . 63
3.3.1 Формулировка задания . . . 63
3.3.2 Вариант № 24 . . . 63
3.3.3 Основные формулы . . . 63
3.3.4 Текст программы . . . 63
3.3.5 График аппроксимации . . . 72
3.3.6 Анализ результатов . . . 72
3.4 Задание 3.4 . . . 72
3.4.1 Формулировка задания . . . 72
3.4.2 Вариант № 24 . . . 72
3.4.3 Основные формулы . . . 72
3.4.4 Текст программы . . . 73
3.4.5 Анализ результатов . . . 77
3.5 Задание 3.5 . . . 77
3.5.1 Формулировка задания . . . 77
3.5.2 Вариант № 24 . . . 77
3.5.3 Основные формулы . . . 77
3.5.4 Текст программы . . . 78
3.5.5 Анализ результатов . . . 84
3.6 Выводы . . . 84

4 Решение дифференциальных уравнений 84
4.1 Задание 4.1 . . . 84
4.1.1 Формулировка задания . . . 84
4.1.2 Вариант № 24 . . . 84
4.1.3 Основные формулы . . . 85
4.1.4 Текст программы . . . 85
4.1.5 Анализ результатов . . . 93
4.2 Задание 4.2 . . . 93
4.2.1 Формулировка задания . . . 93
4.2.2 Вариант № 24 . . . 93
4.2.3 Основные формулы . . . 93
4.2.4 Текст программы . . . 93
4.2.5 Анализ результатов . . . 101
4.3 Выводы . . . 101

5 Итоги 101

1. Решение СЛАУ

1.1. Задание 1.1

1.1.1. Формулировка задания

Реализовать алгоритм LU-разложения матриц с выбором главного элемента. Используя разработанную программу, решить систему линейных алгебраических уравнений, вычислить определитель матрицы системы и обратную матрицу.

1.1.2. Вариант № 24

Решить систему $Ax = b$, где

$$A = \begin{pmatrix} 7 & -5 & 6 & 7 \\ 8 & -1 & -9 & 1 \\ -3 & 8 & 8 & 8 \\ 2 & -3 & 6 & -4 \end{pmatrix}, \qquad b = \begin{pmatrix} 120 \\ 31 \\ 6 \\ 25 \end{pmatrix}.$$

1.1.3. Основные формулы

С учётом перестановок строк разложение записывается в форме

$$PA = LU,$$

где P --- матрица перестановок, L --- нижняя треугольная матрица с единицами на диагонали, U --- верхняя треугольная матрица. Решение находится из двух треугольных систем:

$$Ly = Pb, \qquad Ux = y.$$

Определитель и обратная матрица вычисляются по формулам

$$\det A = (-1)^s \prod_{i=1}^{n} u_{ii}, \qquad A^{-1} = \left(x^{(1)}, \ldots, x^{(n)}\right), \quad Ax^{(j)} = e_j,$$

где s --- число перестановок строк.

1.1.4. Текст программы

```
import numpy as np
import sys



def lu_decomposition_with_pivoting(A: np.ndarray):
    """
    LU-разложение с частичным выбором главного элемента.
    Возвращает P, L, U и число перестановок строк.
    Выполняется: P @ A = L @ U
    """
    A = A.astype(float).copy()   # работа с копией матрицы в float
    n = A.shape[0]

    P = np.eye(n)                # матрица перестановок
    L = np.zeros((n, n))         # нижняя треугольная матрица
    U = A.copy()                 # верхняя треугольная матрица
```

```
    swaps = 0                    # число перестановок строк

    for k in range(n):
        # Ищем главный элемент в текущем столбце
        pivot = np.argmax(np.abs(U[k:, k])) + k

        # Если главный элемент почти ноль, разложение невозможно
        if abs(U[pivot, k]) < 1e-12:
            raise ValueError("Матрица вырождена: LU-разложение невозможно")

        # Переставляем строки, если главный элемент не на диагонали
        if pivot != k:
            U[[k, pivot], :] = U[[pivot, k], :]
            P[[k, pivot], :] = P[[pivot, k], :]

            # Уже найденные коэффициенты в L тоже нужно переставить
            if k > 0:
                L[[k, pivot], :k] = L[[pivot, k], :k]

            swaps += 1

        # В L на диагонали стоят единицы
        L[k, k] = 1.0

        # Зануляем элементы ниже диагонали
        for i in range(k + 1, n):
            L[i, k] = U[i, k] / U[k, k]          # коэффициент исключения
            U[i, k:] -= L[i, k] * U[k, k:]       # вычитание ведущей строки
            U[i, k] = 0.0                        # явно ставим ноль

    return P, L, U, swaps


def forward_substitution(L: np.ndarray, b: np.ndarray):
    """Решение L @ y = b методом прямой подстановки."""
    n = L.shape[0]
    y = np.zeros(n)

    for i in range(n):
        y[i] = b[i] - np.dot(L[i, :i], y[:i])

    return y


def backward_substitution(U: np.ndarray, y: np.ndarray):
    """Решение U @ x = y методом обратной подстановки."""
    n = U.shape[0]
    x = np.zeros(n)

    for i in range(n - 1, -1, -1):
        if abs(U[i, i]) < 1e-12:
            raise ValueError("Нулевой элемент на диагонали U")
        x[i] = (y[i] - np.dot(U[i, i + 1:], x[i + 1:])) / U[i, i]

    return x


def solve_lu(A: np.ndarray, b: np.ndarray):
    """
    Решение системы A @ x = b через LU-разложение:
```

```
    1) P @ A = L @ U
    2) L @ y = P @ b
    3) U @ x = y
    """
    P, L, U, swaps = lu_decomposition_with_pivoting(A)

    Pb = P @ b                              # переставляем правую часть
    y = forward_substitution(L, Pb)         # решаем L @ y = P @ b
    x = backward_substitution(U, y)         # решаем U @ x = y

    # Определитель через диагональ U и число перестановок
    detA = ((-1) ** swaps) * np.prod(np.diag(U))

    return x, P, L, U, detA


def inverse_from_lu(P: np.ndarray, L: np.ndarray, U: np.ndarray):
    """
    Нахождение обратной матрицы через решение систем
    для столбцов единичной матрицы.
    """
    n = L.shape[0]
    A_inv = np.zeros((n, n))

    for j in range(n):
        e = np.zeros(n)
        e[j] = 1.0                           # j-й столбец единичной матрицы

        y = forward_substitution(L, P @ e)
        col = backward_substitution(U, y)

        A_inv[:, j] = col                   # записываем столбец обратной матрицы

    return A_inv


if __name__ == "__main__":
    # Матрица и вектор для варианта 24
    A = np.array([
        [7, -5, 6, 7],
        [8, -1, -9, 1],
        [-3, 8, 8, 8],
        [2, -3, 6, -4]
    ], dtype=float)

    b = np.array([120, 31, 6, 25], dtype=float)

    # Решение системы
    x, P, L, U, detA = solve_lu(A, b)

    # Обратная матрица
    A_inv = inverse_from_lu(P, L, U)

    np.set_printoptions(precision=6, suppress=True)

    print("Исходная матрица A:")
    print(A)

    print("\nВектор b:")
    print(b)
```

```
    print("\nМатрица перестановок P:")
    print(P)

    print("\nМатрица L:")
    print(L)

    print("\nМатрица U:")
    print(U)

    print("\nПроверка P @ A:")
    print(P @ A)

    print("\nПроверка L @ U:")
    print(L @ U)

    print("\nРешение системы x:")
    print(x)

    print("\nПроверка A @ x:")
    print(A @ x)

    print("\nОпределитель det(A):")
    print(detA)

    print("\nОбратная матрица A^(-1):")
    print(A_inv)

    print("\nПроверка A @ A^(-1):")
    print(A @ A_inv)
```

Вывод программы:

```
Исходная матрица A:
[[ 7. -5.  6.  7.]
 [ 8. -1. -9.  1.]
 [-3.  8.  8.  8.]
 [ 2. -3.  6. -4.]]

Вектор b:
[120.  31.   6.  25.]

Матрица перестановок P:
[[0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [1. 0. 0. 0.]
 [0. 0. 0. 1.]]

Матрица L:
[[ 1.        0.        0.        0.      ]
 [-0.375     1.        0.        0.      ]
 [ 0.875    -0.540984  1.        0.      ]
 [ 0.25     -0.360656  0.605606  1.      ]]

Матрица U:
[[ 8.       -1.       -9.        1.      ]
 [ 0.        7.625     4.625     8.375   ]
 [ 0.        0.       16.377049 10.655738]
 [ 0.        0.        0.       -7.682683]]
```

```
Проверка P @ A:
[[ 8. -1. -9.  1.]
 [-3.  8.  8.  8.]
 [ 7. -5.  6.  7.]
 [ 2. -3.  6. -4.]]

Проверка L @ U:
[[ 8. -1. -9.  1.]
 [-3.  8.  8.  8.]
 [ 7. -5.  6.  7.]
 [ 2. -3.  6. -4.]]

Решение системы x:
[ 6. -5.  3.  5.]

Проверка A @ x:
[120.  31.   6.  25.]

Определитель det(A):
-7674.999999999999

Обратная матрица A^(-1):
[[-0.010423  0.124039  0.055114  0.122997]
 [-0.092508  0.100847  0.114137  0.091596]
 [ 0.009772 -0.016287  0.035831  0.084691]
 [ 0.078827 -0.038046 -0.0043   -0.130163]]

Проверка A @ A^(-1):
[[ 1.  0.  0.  0.]
 [ 0.  1.  0.  0.]
 [ 0.  0.  1.  0.]
 [-0.  0.  0.  1.]]
```

1.2. Задание 1.2

1.2.1. Формулировка задания

Реализовать метод прогонки для решения СЛАУ с трёхдиагональной матрицей, задавая ненулевые диагонали матрицы и вектор правых частей.

1.2.2. Вариант № 24

Ненулевые диагонали и правая часть системы имеют вид

$$
\begin{aligned}
a &= (0,\ 2,\ -7,\ 2,\ -7),\\
b &= (-14,\ 7,\ -18,\ -13,\ -7),\\
c &= (6,\ 0,\ -9,\ 2,\ 0),\\
d &= (82,\ -51,\ -46,\ 111,\ 35).
\end{aligned}
$$

Здесь a, b, c --- соответственно нижняя, главная и верхняя диагонали.

1.2.3. Основные формулы

Для трёхдиагональной системы

$$a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$$

решение ищется в форме $x_i = P_i x_{i+1} + Q_i$. Прямой ход:

$$P_1 = -\frac{c_1}{b_1}, \quad Q_1 = \frac{d_1}{b_1}, \qquad P_i = -\frac{c_i}{b_i + a_i P_{i-1}}, \quad Q_i = \frac{d_i - a_i Q_{i-1}}{b_i + a_i P_{i-1}}.$$

Обратный ход начинается с $x_n = Q_n$ и выполняется по формуле

$$x_i = P_i x_{i+1} + Q_i, \qquad i = n-1, \ldots, 1.$$

1.2.4. Текст программы

```
def check_stability(a, b, c):
    """
    Проверка достаточных условий устойчивости метода прогонки.
    """

    n = len(b)

    # Проверка внутренних коэффициентов:
    # для внутренних строк под- и наддиагональные элементы должны быть
    ↪ ненулевыми
    for i in range(1, n - 1):
        if a[i] == 0:
            raise ValueError(
                f"Нарушено условие устойчивости: a[{i}] = 0 "
                f"(для внутренней строки поддиагональный элемент должен быть
                ↪ ненулевым)."
            )
        if c[i] == 0:
            raise ValueError(
                f"Нарушено условие устойчивости: c[{i}] = 0 "
                f"(для внутренней строки наддиагональный элемент должен быть
                ↪ ненулевым)."
            )

    # Проверка диагонального преобладания
    strict_found = False  # будет True, если хотя бы в одной строке неравенство
    ↪ строгое

    for i in range(n):
        if abs(b[i]) < abs(a[i]) + abs(c[i]):
            raise ValueError(
                f"Нарушено условие устойчивости в строке {i + 1}: "
                f"|b[{i}]| = {abs(b[i])} < |a[{i}]| + |c[{i}]| = {abs(a[i]) +
                ↪ abs(c[i])}"
            )

        if abs(b[i]) > abs(a[i]) + abs(c[i]):
            strict_found = True

    if not strict_found:
        raise ValueError(
            "Нарушено условие устойчивости: ни в одной строке нет строгого "
            "диагонального преобладания."
        )

def thomas_algorithm(a, b, c, d):
    """
    Метод прогонки для решения трёхдиагональной СЛАУ.
```

```
    a — поддиагональ
    b — главная диагональ
    c — наддиагональ
    d — правая часть
    """

    # Число уравнений
    n = len(d)

    # Прогоночные коэффициенты:
    # x[i] = P[i] * x[i+1] + Q[i]
    P = [0.0] * n
    Q = [0.0] * n

    # Начальные коэффициенты по первой строке
    P[0] = -c[0] / b[0]
    Q[0] = d[0] / b[0]

    # Прямой ход: вычисление P[i] и Q[i]
    for i in range(1, n):
        denominator = b[i] + a[i] * P[i - 1]

        # Проверка на деление на ноль
        if abs(denominator) < 1e-12:
            raise ValueError(f"Нулевой знаменатель на шаге {i}")

        # Для последней строки P[i] не нужен
        if i < n - 1:
            P[i] = -c[i] / denominator
        else:
            P[i] = 0.0

        Q[i] = (d[i] - a[i] * Q[i - 1]) / denominator

    # Обратный ход: восстановление решения
    x = [0.0] * n
    x[n - 1] = Q[n - 1]

    for i in range(n - 2, -1, -1):
        x[i] = P[i] * x[i + 1] + Q[i]

    return x


# Вариант 24
a = [0, 2, -7, 2, -7]          # поддиагональ
b = [-14, 7, -18, -13, -7]     # главная диагональ
c = [6, 0, -9, 2, 0]           # наддиагональ
d = [82, -51, -46, 111, 35]    # правая часть

solution = thomas_algorithm(a, b, c, d)

print("Решение системы:")
for i, xi in enumerate(solution, start=1):
    print(f"x{i} = {xi}")
```

Вывод программы:

```
Решение системы:
x1 = -8.0
```

```
x2 = -5.0
x3 = 8.0
x4 = -7.0
x5 = 2.0
```

1.3. Задание 1.3

1.3.1. Формулировка задания

Реализовать методы простых итераций и Зейделя для решения СЛАУ. Проанализировать число итераций, необходимое для достижения заданной точности.

1.3.2. Вариант № 24

$$A = \begin{pmatrix} -18 & 9 & -1 & -8 \\ 6 & 22 & 9 & 0 \\ -4 & 2 & -16 & 9 \\ 1 & 6 & -1 & -14 \end{pmatrix}, \qquad b = \begin{pmatrix} -60 \\ -109 \\ -103 \\ -33 \end{pmatrix}, \qquad \varepsilon = 10^{-3}.$$

1.3.3. Основные формулы

Систему приводим к виду

$$x = \beta + \alpha x, \qquad \beta_i = \frac{b_i}{a_{ii}}, \qquad \alpha_{ij} = \begin{cases} -a_{ij}/a_{ii}, & i \neq j, \\ 0, & i = j. \end{cases}$$

Формула метода Якоби:

$$x_i^{(k+1)} = \beta_i + \sum_{j=1}^{n} \alpha_{ij} x_j^{(k)}.$$

Формула метода Зейделя:

$$x_i^{(k+1)} = \beta_i + \sum_{j=1}^{i-1} \alpha_{ij} x_j^{(k+1)} + \sum_{j=i+1}^{n} \alpha_{ij} x_j^{(k)}.$$

1.3.4. Текст программы

```
import numpy as np


def vector_c_norm(x):
    """
    c-норма вектора:
        ||x||_c = max_i |x_i|

    В методичке именно эта норма используется как одна из основных
    для исследования сходимости и погрешности.
    """
    return np.max(np.abs(x))


def matrix_c_norm(A):
    """
    c-норма матрицы:
```

```
        ||A||_c = max_i sum_j |a_ij|

    Это максимум суммы модулей элементов по строкам.
    В методичке такая норма приведена как согласованная с c-нормой вектора.
    """
    return np.max(np.sum(np.abs(A), axis=1))


def check_diagonal_dominance(A):
    """
    Проверка диагонального преобладания по строкам.

    Для каждой строки сравниваем:
        |a_ii|  и  sum_{j != i} |a_ij|

    Возвращаем два логических значения:
    - strict_ok: строгое диагональное преобладание по всем строкам
    - weak_ok:   нестрогое диагональное преобладание по всем строкам
    """
    n = A.shape[0]
    strict_ok = True
    weak_ok = True

    for i in range(n):
        diag = abs(A[i, i])
        off_diag_sum = np.sum(np.abs(A[i])) - diag

        if diag < off_diag_sum:
            weak_ok = False
            strict_ok = False
        elif diag == off_diag_sum:
            strict_ok = False

    return strict_ok, weak_ok


def build_beta_alpha(A, b):
    """
    Строим эквивалентную систему вида:
        x = beta + alpha * x

    По методичке:
        beta_i  = b_i / a_ii
        alpha_ij = -a_ij / a_ii,  если i != j
        alpha_ii = 0

    Если на диагонали есть нулевой элемент, метод в таком виде применять нельзя:
    нужно переставить уравнения местами.
    """
    n = A.shape[0]
    beta = np.zeros(n, dtype=float)
    alpha = np.zeros((n, n), dtype=float)

    for i in range(n):
        if A[i, i] == 0:
            raise ValueError(
                f"Нулевой диагональный элемент A[{i}][{i}]. "
                "Нужно переставить уравнения местами."
            )
```

```
        beta[i] = b[i] / A[i, i]

        for j in range(n):
            if i != j:
                alpha[i, j] = -A[i, j] / A[i, i]

    return beta, alpha


def print_method_info(name, A, alpha):
    """
    Печатаем теоретическую информацию перед запуском метода:
    - есть ли диагональное преобладание;
    - чему равна ||alpha||_c;
    - какой критерий остановки будет использоваться.
    """
    strict_dom, weak_dom = check_diagonal_dominance(A)
    alpha_norm = matrix_c_norm(alpha)

    print(f"\n{name}")
    print("-" * len(name))

    if strict_dom:
        print("Диагональное преобладание: СТРОГОЕ.")
        print("Это достаточное условие сходимости.")
    elif weak_dom:
        print("Диагональное преобладание: НЕСТРОГОЕ.")
        print("Строгой гарантии по этому признаку нет.")
    else:
        print("Диагонального преобладания нет.")
        print("Гарантия сходимости по этому признаку отсутствует.")

    print(f"||alpha||_c = {alpha_norm:.6f}")

    if alpha_norm < 1:
        print("Условие ||alpha||_c < 1 выполнено.")
        print("Можно использовать теоретическую оценку погрешности.")
    else:
        print("Условие ||alpha||_c < 1 НЕ выполнено.")
        print("Используем практический критерий остановки по разности соседних
        ↪  приближений.")


def jacobi(A, b, eps=1e-6, max_iter=10000):
    """
    Метод простых итераций в форме Якоби.

    Сначала строим эквивалентную систему:
        x = beta + alpha * x

    Затем запускаем итерационный процесс:
        x^(k+1) = beta + alpha * x^(k)

    В методичке указано:
    - если ||alpha|| < 1, можно использовать оценку погрешности;
    - если это условие не выполнено, но метод все же сходится,
      можно завершать итерации по условию:
          ||x^(k+1) - x^(k)|| < eps
    """
    beta, alpha = build_beta_alpha(A, b)
```

```
    alpha_norm = matrix_c_norm(alpha)

    # В качестве начального приближения берем beta,
    # как это предлагается в методичке.
    x_old = beta.copy()

    print_method_info("Метод простых итераций (Якоби)", A, alpha)

    for k in range(1, max_iter + 1):
        # Формула метода Якоби:
        x_new = beta + alpha @ x_old
        # Если выполнено достаточное условие ||alpha|| < 1,
        # можно использовать оценку погрешности
        # Если она меньше заданной точности, процесс завершаем.
        if alpha_norm < 1:
            est_error = alpha_norm / (1 - alpha_norm) * vector_c_norm(x_new -
            ↪  x_old)
            if est_error < eps:
                return x_new, k
        else:
            # Если ||alpha|| >= 1, используем практический критерий:
            # разность соседних приближений стала меньше eps.
            if vector_c_norm(x_new - x_old) < eps:
                return x_new, k

        x_old = x_new.copy()

    raise ValueError("Метод простых итераций не сошелся за max_iter итераций")


def gauss_seidel(A, b, eps=1e-6, max_iter=10000):
    """
    Метод Зейделя.
    Так же, как и в методе Якоби, сначала строим эквивалентную систему:
        x = beta + alpha * x

    Но при вычислении i-й компоненты на текущей итерации:
    - для j < i используем уже обновленные значения x[j];
    - для j > i используем старые значения x_old[j].
    """
    beta, alpha = build_beta_alpha(A, b)
    alpha_norm = matrix_c_norm(alpha)

    # Начальное приближение также берем равным beta.
    x = beta.copy()

    print_method_info("Метод Зейделя", A, alpha)

    for k in range(1, max_iter + 1):
        x_old = x.copy()

        # Последовательно пересчитываем компоненты решения.
        for i in range(len(b)):
            # Формула Зейделя для i-й компоненты:
            # x_i^(k+1) = beta_i
            #             + sum_{j < i} alpha_ij * x_j^(k+1)
            #             + sum_{j > i} alpha_ij * x_j^(k)
            s1 = np.dot(alpha[i, :i], x[:i])          # уже обновленные значения
            s2 = np.dot(alpha[i, i + 1:], x_old[i + 1:])  # старые значения
            x[i] = beta[i] + s1 + s2
```

```
        if alpha_norm < 1:
            # В методичке для Зейделя при выполнении достаточных условий
            # тоже используют оценку погрешности.
            # Здесь для практической реализации берем ту же схему:
           est_error = alpha_norm / (1 - alpha_norm) * vector_c_norm(x - x_old)
            if est_error < eps:
                return x, k
        else:
            if vector_c_norm(x - x_old) < eps:
                return x, k

    raise ValueError("Метод Зейделя не сошелся за max_iter итераций")


def print_solution(title, x, iters):
    """
    Красивый вывод найденного решения и числа итераций.
    """
    print(f"\n{title}:")
    for i, value in enumerate(x, start=1):
        print(f"x{i} = {value:.6f}")
    print(f"Количество итераций: {iters}")


def main():
    # Матрица коэффициентов системы A * x = b
    A = np.array([
        [-18,  9, -1, -8],
        [  6, 22,  9,  0],
        [ -4,  2, -16,  9],
        [  1,  6, -1, -14]
    ], dtype=float)

    # Вектор правых частей
    b = np.array([-60, -109, -103, -33], dtype=float)

    # Заданная пользователем точность
    eps = float(input("Введите точность eps (например, 0.001): "))

    # Решаем систему двумя итерационными методами
    x_jacobi, iter_jacobi = jacobi(A, b, eps)
    x_seidel, iter_seidel = gauss_seidel(A, b, eps)

    # Выводим результаты
    print_solution("Результат метода простых итераций (Якоби)", x_jacobi,
    ↪  iter_jacobi)
    print_solution("Результат метода Зейделя", x_seidel, iter_seidel)


# Стандартная точка входа в программу
if __name__ == "__main__":
    main()
```

Вывод программы:

```
Введите точность eps (например, 0.001):
Метод простых итераций (Якоби)
------------------------------
```

```
Диагональное преобладание: НЕСТРОГОЕ.
Строгой гарантии по этому признаку нет.
||alpha||_c = 1.000000
Условие ||alpha||_c < 1 НЕ выполнено.
Используем практический критерий остановки по разности соседних приближений.

Метод Зейделя
-------------
Диагональное преобладание: НЕСТРОГОЕ.
Строгой гарантии по этому признаку нет.
||alpha||_c = 1.000000
Условие ||alpha||_c < 1 НЕ выполнено.
Используем практический критерий остановки по разности соседних приближений.

Результат метода простых итераций (Якоби):
x1 = -0.000203
x2 = -7.000044
x3 = 5.000203
x4 = -1.000065
Количество итераций: 12

Результат метода Зейделя:
x1 = 0.000255
x2 = -6.999996
x3 = 5.000129
x4 = -0.999989
Количество итераций: 8
```

1.3.5. Анализ результатов

Для реализованной системы программа получила $\|\alpha\|_\infty = 1$, поэтому в качестве практического критерия остановки использована норма разности соседних приближений. Метод Якоби завершился за 12 итераций, метод Зейделя --- за 8 итераций. В данной задаче использование новых компонент в пределах одной итерации ускоряет сходимость метода Зейделя.

1.4. Задание 1.4

1.4.1. Формулировка задания

Реализовать метод вращений Якоби. Найти собственные значения и собственные векторы симметричной матрицы, а также исследовать уменьшение погрешности при выполнении итераций.

1.4.2. Вариант № 24

$$A = \begin{pmatrix} 2 & -9 & 4 \\ -9 & 0 & 9 \\ 4 & 9 & 6 \end{pmatrix}, \qquad \varepsilon = 10^{-3}.$$

1.4.3. Основные формулы

На каждом шаге выполняется ортогональное преобразование подобия

$$A^{(k+1)} = U_k^T A^{(k)} U_k,$$

где матрица U_k выбирается так, чтобы обнулить максимальный по модулю внедиагональный элемент a_{pq}. Угол вращения определяется выражением

$$\varphi = \frac{1}{2} \arctan \frac{2a_{pq}}{a_{pp} - a_{qq}}.$$

Критерием остановки служит значение

$$t(A) = \sqrt{\sum_{i<j} a_{ij}^2} < \varepsilon.$$

1.4.4. Текст программы

```
import math


def print_matrix(M, name="Matrix"):
    """
    Вывод матрицы на экран.
    """
    print(f"\n{name}:")
    for row in M:
        print(" ".join(f"{x:12.6f}" for x in row))


def identity_matrix(n):
    """
    Построение единичной матрицы размера n x n.
    """
    return [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]


def transpose(A):
    """
    Транспонирование матрицы A.
    """
    n = len(A)
    m = len(A[0])
    return [[A[i][j] for i in range(n)] for j in range(m)]


def multiply_matrices(A, B):
    """
    Умножение двух матриц A и B.
    """
    n = len(A)
    m = len(B[0])
    k = len(B)
    C = [[0.0 for _ in range(m)] for _ in range(n)]

    for i in range(n):
        for j in range(m):
            for t in range(k):
                C[i][j] += A[i][t] * B[t][j]

    return C


def is_symmetric(A, tol=1e-12):
```

```
    """
    Проверка симметричности матрицы.
    Метод вращений Якоби применим только к симметрическим матрицам.
    """
    n = len(A)
    for i in range(n):
        for j in range(n):
            if abs(A[i][j] - A[j][i]) > tol:
                return False
    return True


def off_diagonal_sum_squares(A):
    """
    Сумма квадратов внедиагональных элементов матрицы.
    Этот критерий используется для остановки итераций.
    """
    n = len(A)
    s = 0.0

    for i in range(n):
        for j in range(n):
            if i != j:
                s += A[i][j] ** 2

    return s


def off_diagonal_norm(A):
    """
    Корень из суммы квадратов внедиагональных элементов.
    Используется для вывода погрешности по итерациям.
    """
    return math.sqrt(off_diagonal_sum_squares(A))


def max_offdiag_index(A):
    """
    Поиск максимального по модулю внедиагонального элемента матрицы.
    Возвращает его индексы (p, q).
    """
    n = len(A)
    max_val = abs(A[0][1])
    p, q = 0, 1

    for i in range(n):
        for j in range(i + 1, n):
            if abs(A[i][j]) > max_val:
                max_val = abs(A[i][j])
                p, q = i, j

    return p, q


def jacobi_rotation_method(A, eps=1e-6, max_iter=100):
    """
    Метод вращений Якоби для нахождения собственных значений
    и собственных векторов симметрической матрицы.
    """
```

```
n = len(A)

if not is_symmetric(A):
    raise ValueError(
        "Метод вращений Якоби применим только к симметрическим матрицам."
    )

# Начальная матрица A^(0)
A_k = [row[:] for row in A]

# Накопленная матрица вращений
U = identity_matrix(n)

# История изменения погрешности
history = []

for iteration in range(max_iter):
    off_sq = off_diagonal_sum_squares(A_k)
    err = math.sqrt(off_sq)

    history.append((iteration, err, [A_k[i][i] for i in range(n)]))

    # Критерий остановки:
    # сумма квадратов внедиагональных элементов должна стать меньше eps^2
    if off_sq < eps ** 2:
        break

    # Выбираем максимальный по модулю внедиагональный элемент
    i, j = max_offdiag_index(A_k)

    # Вычисляем угол поворота
    if A_k[i][i] == A_k[j][j]:
        phi = math.pi / 4
    else:
        phi = 0.5 * math.atan((2 * A_k[i][j]) / (A_k[i][i] - A_k[j][j]))

    c = math.cos(phi)
    s = math.sin(phi)

    # Строим матрицу вращения U_k
    R = identity_matrix(n)
    R[i][i] = c
    R[j][j] = c
    R[i][j] = -s
    R[j][i] = s

    # Выполняем преобразование подобия:
    # A^(k+1) = U_k^T * A^(k) * U_k
    Rt = transpose(R)
    A_k = multiply_matrices(multiply_matrices(Rt, A_k), R)

    # Накапливаем произведение матриц вращений
    U = multiply_matrices(U, R)

# Диагональные элементы итоговой матрицы - собственные значения
eigenvalues = [A_k[i][i] for i in range(n)]

# Столбцы матрицы U - собственные векторы
eigenvectors = [[U[i][j] for i in range(n)] for j in range(n)]
```

```
    return eigenvalues, eigenvectors, history, A_k


def main():
    """
    Основная функция программы.
    """

    A = [
        [2.0, -9.0, 4.0],
        [-9.0, 0.0, 9.0],
        [4.0, 9.0, 6.0]
    ]

    eps = float(input("Введите точность eps (например, 0.001): "))

    eigenvalues, eigenvectors, history, A_final = jacobi_rotation_method(A,
    ↪  eps)

    print_matrix(A, "Исходная матрица")
    print_matrix(A_final, "Почти диагональная матрица после вращений")

    print("\nСобственные значения:")
    for i, lam in enumerate(eigenvalues, start=1):
        print(f"lambda{i} = {lam:.6f}")

    print("\nСобственные векторы:")
    for i, vec in enumerate(eigenvectors, start=1):
        print(f"x{i} = (" + ", ".join(f"{v:.6f}" for v in vec) + ")")

    print("\nЗависимость погрешности от числа итераций:")
    print(f"{'Итерация':>10} {'Погрешность':>15}")
    for it, err, diag in history:
        print(f"{it:10d} {err:15.8f}")

    print(f"\nКоличество итераций: {len(history) - 1}")


if __name__ == "__main__":
    main()
```

Вывод программы:

```
Введите точность eps (например, 0.001):
Исходная матрица:
    2.000000    -9.000000     4.000000
   -9.000000     0.000000     9.000000
    4.000000     9.000000     6.000000

Почти диагональная матрица после вращений:
    7.700117     0.000000    -0.000000
    0.000000   -12.824326     0.000013
   -0.000000     0.000013    13.124209

Собственные значения:
lambda1 = 7.700117
lambda2 = -12.824326
lambda3 = 13.124209
```

```
Собственные векторы:
x1 = (0.781024, -0.238060, 0.577347)
x2 = (0.548811, 0.702803, -0.452630)
x3 = (-0.298009, 0.670369, 0.679556)

Зависимость погрешности от числа итераций:
  Итерация     Погрешность
         0     18.86796226
         1     13.92838828
         2      4.27344380
         3      1.91149268
         4      1.27064532
         5      0.06268364
         6      0.00216914
         7      0.00001772

Количество итераций: 7
```

1.4.5. Анализ результатов

За 7 итераций норма внедиагональных элементов уменьшилась до $1.772 \cdot 10^{-5}$. Диагональные элементы преобразованной матрицы дают приближения собственных значений 7.700117, -12.824326 и 13.124209.

1.5. Задание 1.5

1.5.1. Формулировка задания

Реализовать QR-разложение матрицы методом отражений Хаусхолдера и QR-алгоритм нахождения собственных значений произвольной матрицы.

1.5.2. Вариант № 24

$$A = \begin{pmatrix} -1 & 8 & 5 \\ 8 & -4 & 4 \\ 2 & 9 & -2 \end{pmatrix}, \qquad \varepsilon = 10^{-3}.$$

1.5.3. Основные формулы

QR-разложение имеет вид

$$A = QR, \qquad Q^T Q = E,$$

где Q --- ортогональная матрица, а R --- верхняя треугольная матрица. Отражение Хаусхолдера строится по формуле

$$H = E - 2\frac{vv^T}{v^T v}.$$

Итерации алгоритма собственных значений:

$$A^{(k)} = Q^{(k)} R^{(k)}, \qquad A^{(k+1)} = R^{(k)} Q^{(k)}.$$

1.5.4. Текст программы

```
import math
import cmath


# ============================================================
# ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ
# ============================================================

def print_matrix(A, title="Матрица"):
    """Красивый вывод матрицы."""
    print(title + ":")
    for row in A:
        print(" ".join(f"{x:12.6f}" for x in row))
    print()


def identity_matrix(n):
    """Единичная матрица размера n x n."""
    return [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]


def copy_matrix(A):
    """Копия матрицы."""
    return [row[:] for row in A]


def transpose(A):
    """Транспонирование матрицы."""
    n = len(A)
    m = len(A[0])
    return [[A[i][j] for i in range(n)] for j in range(m)]


def matmul(A, B):
    """Умножение матриц A * B."""
    n = len(A)
    m = len(B[0])
    k = len(B)

    C = [[0.0 for _ in range(m)] for _ in range(n)]
    for i in range(n):
        for j in range(m):
            s = 0.0
            for t in range(k):
                s += A[i][t] * B[t][j]
            C[i][j] = s
    return C


def vector_norm_2(v):
    """Евклидова норма вектора."""
    return math.sqrt(sum(x * x for x in v))


def is_square_matrix(A):
    """Проверка, что матрица квадратная."""
    if not isinstance(A, list) or len(A) == 0:
        return False
    n = len(A)
```

```
    return all(isinstance(row, list) and len(row) == n for row in A)


def sign(x):
    """
    Знак числа.
    В методичке используется sign(b1).
    """
    return 1.0 if x >= 0 else -1.0


# ============================================================
# ХАУСХОЛДЕР: ВЕКТОР И МАТРИЦА
# ============================================================

def build_householder_vector(b):
    """
    Построение вектора Хаусхолдера по формуле:

        v = b + sign(b1) * ||b|| * e1
    """
    norm_b = vector_norm_2(b)
    if norm_b < 1e-15:
        return None

    v = b[:]
    v[0] += sign(b[0]) * norm_b

    norm_v = vector_norm_2(v)
    if norm_v < 1e-15:
        return None

    return v


def build_householder_matrix(v, n, start):
    """
    Строим полную матрицу Хаусхолдера размера n x n.

    Локальная формула:
        H_local = E - 2 * (v v^T) / (v^T v)

    Затем встраиваем ее в общую матрицу с позиции 'start'.
    """
    H = identity_matrix(n)
    m = len(v)

    vt_v = sum(x * x for x in v)
    if abs(vt_v) < 1e-15:
        return H

    for i in range(m):
        for j in range(m):
            H[start + i][start + j] -= 2.0 * v[i] * v[j] / vt_v

    return H


# ============================================================
# QR-РАЗЛОЖЕНИЕ МЕТОДОМ ХАУСХОЛДЕРА
```

```
# ============================================================

def householder_qr(A):
    """
    QR-разложение матрицы A:
        A = Q * R

    По методичке:
    последовательно строим матрицы Хаусхолдера H1, H2, ...,
    зануляя элементы ниже главной диагонали в каждом столбце.
    """
    if not is_square_matrix(A):
        raise ValueError("Матрица должна быть квадратной.")

    n = len(A)
    R = copy_matrix(A)
    Q = identity_matrix(n)

    # На k-м шаге зануляем элементы ниже диагонали в k-м столбце
    for k in range(n - 1):
        # Берем подстолбец начиная с диагонального элемента
        b = [R[i][k] for i in range(k, n)]

        v = build_householder_vector(b)
        if v is None:
            continue

        H = build_householder_matrix(v, n, k)

        # Преобразуем текущую матрицу
        R = matmul(H, R)

        # Накапливаем итоговую ортогональную матрицу Q
        Q = matmul(Q, H)

    return Q, R


# ============================================================
# ПРИВЕДЕНИЕ К ВЕРХНЕЙ ФОРМЕ ХЕССЕНБЕРГА
# ============================================================

def hessenberg_reduction(A):
    """
    Предварительное приведение матрицы к верхней форме Хессенберга.

    По методичке перед QR-алгоритмом матрицу полезно привести
    к виду A^(0) = H^T * A * H, где A^(0) - верхняя хессенбергова.

    В верхней форме Хессенберга все элементы ниже первой поддиагонали равны
    ↪ нулю:
        a[i][j] = 0, если i > j + 1
    """
    if not is_square_matrix(A):
        raise ValueError("Матрица должна быть квадратной.")

    n = len(A)
    H_total = identity_matrix(n)
    A_h = copy_matrix(A)
```

```
    # Нужно занулять элементы ниже первой поддиагонали
    for k in range(n - 2):
        # Берем часть столбца ниже первой поддиагонали:
        # a[k+1][k], a[k+2][k], ..., a[n-1][k]
        b = [A_h[i][k] for i in range(k + 1, n)]

        v = build_householder_vector(b)
        if v is None:
            continue

        H = build_householder_matrix(v, n, k + 1)

        # Так как H - симметричная и ортогональная:
        # H^T = H
        A_h = matmul(matmul(H, A_h), H)
        H_total = matmul(H_total, H)

    return A_h, H_total


def is_hessenberg(A, tol=1e-10):
    """Проверка, что матрица верхняя хессенбергова."""
    n = len(A)
    for i in range(n):
        for j in range(n):
            if i > j + 1 and abs(A[i][j]) > tol:
                return False
    return True


# ============================================================
# ВСПОМОГАТЕЛЬНОЕ ДЛЯ QR-АЛГОРИТМА
# ============================================================

def solve_quadratic_for_2x2_block(a, b, c, d):
    """
    Собственные значения блока 2x2:
        | a  b |
        | c  d |

    Характеристическое уравнение:
        lambda^2 - (a+d) lambda + (ad - bc) = 0
    """
    trace = a + d
    det = a * d - b * c
    discriminant = trace * trace - 4.0 * det

    sqrt_disc = cmath.sqrt(discriminant)
    lambda1 = (trace + sqrt_disc) / 2.0
    lambda2 = (trace - sqrt_disc) / 2.0
    return lambda1, lambda2


# ============================================================
# QR-АЛГОРИТМ ДЛЯ СОБСТВЕННЫХ ЗНАЧЕНИЙ
# ============================================================

def qr_eigenvalues(A, eps=1e-6, max_iter=1000):
    """
    QR-алгоритм по методичке.
```

```
Этапы:
1. Приведение A к форме Хессенберга: A^(0)
2. Далее итерации:
       A^(k) = Q^(k) R^(k)
       A^(k+1) = R^(k) Q^(k)
3. Постепенно выделяем собственные значения:
   - если поддиагональный элемент a[n-1][n-2] мал,
     считаем a[n-1][n-1] вещественным собственным значением;
   - иначе работаем с нижним блоком 2x2, который может дать
     два вещественных или комплексно-сопряженных собственных значения.

Возвращаем:
- список собственных значений,
- итоговую матрицу после итераций,
- число итераций,
- матрицу A^(0) после приведения к Хессенбергу.
"""
if not is_square_matrix(A):
    raise ValueError("Матрица должна быть квадратной.")
if eps <= 0:
    raise ValueError("eps должен быть > 0.")
if max_iter <= 0:
    raise ValueError("max_iter должен быть > 0.")

n = len(A)

# Шаг 1. Приведение к форме Хессенберга
A0, _ = hessenberg_reduction(A)
Ak = copy_matrix(A0)

eigenvalues = []
total_iterations = 0

# Размер активного верхнего левого блока,
# в котором еще не выделены все собственные значения
m = n

while m > 0:
    # Если остался блок 1x1, его диагональный элемент и есть собственное
    ↪ значение
    if m == 1:
        eigenvalues.append(Ak[0][0])
        break

    # Если нижний поддиагональный элемент мал, то выделяем вещественное
    ↪ собственное значение
    if abs(Ak[m - 1][m - 2]) < eps:
        eigenvalues.append(Ak[m - 1][m - 1])
        m -= 1
        continue

    # Если остался блок 2x2, вычисляем его собственные значения напрямую
    if m == 2:
        a = Ak[0][0]
        b = Ak[0][1]
        c = Ak[1][0]
        d = Ak[1][1]
        lambda1, lambda2 = solve_quadratic_for_2x2_block(a, b, c, d)
        eigenvalues.append(lambda1)
```

```
            eigenvalues.append(lambda2)
            break

        # QR-итерации на активном блоке размера m x m
        converged = False
        for _ in range(max_iter):
            total_iterations += 1

            # Берем активный ведущий блок m x m
            active_block = [row[:m] for row in Ak[:m]]

            # QR-разложение активного блока
            Q, R = householder_qr(active_block)

            # A^(k+1) = R^(k) Q^(k)
            next_block = matmul(R, Q)

            # Возвращаем обновленный блок на место
            for i in range(m):
                for j in range(m):
                    Ak[i][j] = next_block[i][j]

            # Проверяем, не отделилось ли 1x1 снизу
            if abs(Ak[m - 1][m - 2]) < eps:
                converged = True
                break

            # Проверяем, не образовался ли 2x2 блок снизу
            # Это не строгая "детекция комплексности", но позволяет корректно
            ↪  завершать
            # процесс на последнем блоке 2x2.
            if abs(Ak[m - 2][m - 3]) < eps:
                converged = True
                break

        if not converged:
            raise RuntimeError(
                "QR-алгоритм не сошелся за заданное число итераций. "
                "Попробуйте увеличить max_iter или ослабить eps."
            )

    return eigenvalues, Ak, total_iterations, A0


# ============================================================
# ОСНОВНАЯ ПРОГРАММА
# ============================================================

def main():
    # Вариант 24
    A = [
        [-1.0, 8.0, 5.0],
        [8.0, -4.0, 4.0],
        [2.0, 9.0, -2.0]
    ]

    eps = float(input("Введите точность eps (например, 0.001): "))

    print_matrix(A, "Исходная матрица")
```

```
    # ------------------------------------------------------
    # 1. QR-разложение исходной матрицы
    # ------------------------------------------------------
    Q, R = householder_qr(A)
    print_matrix(Q, "Матрица Q")
    print_matrix(R, "Матрица R")
    print_matrix(matmul(Q, R), "Проверка Q * R")

    # Проверка ортогональности Q
    QtQ = matmul(transpose(Q), Q)
    print_matrix(QtQ, "Проверка ортогональности Q^T * Q")

    # ------------------------------------------------------
    # 2. Приведение к форме Хессенберга
    # ------------------------------------------------------
    A0, _ = hessenberg_reduction(A)
    print_matrix(A0, "Матрица A^(0) после приведения к форме Хессенберга")
    print(f"Проверка формы Хессенберга: {is_hessenberg(A0)}")
    print()

    # ------------------------------------------------------
    # 3. QR-алгоритм
    # ------------------------------------------------------
    eigenvalues, Ak, iterations, _ = qr_eigenvalues(A, eps=eps, max_iter=1000)

    print_matrix(Ak, "Матрица после QR-итераций")

    print("Собственные значения:")
    for i, lam in enumerate(eigenvalues, start=1):
        if abs(lam.imag) < 1e-12:
            print(f"lambda{i} = {lam.real:.6f}")
        else:
            print(f"lambda{i} = {lam}")

    print(f"\nКоличество QR-итераций: {iterations}")


if __name__ == "__main__":
    main()
```

Вывод программы:

```
Введите точность eps (например, 0.001): Исходная матрица:
   -1.000000     8.000000     5.000000
    8.000000    -4.000000     4.000000
    2.000000     9.000000    -2.000000

Матрица Q:
   -0.120386    -0.618995    -0.776114
    0.963087     0.116791    -0.242536
    0.240772    -0.776663     0.582086

Матрица R:
    8.306624    -2.648489     2.768875
    0.000000   -12.409090    -1.074481
    0.000000    -0.000000    -6.014884

Проверка Q * R:
   -1.000000     8.000000     5.000000
```

```
    8.000000    -4.000000     4.000000
    2.000000     9.000000    -2.000000

Проверка ортогональности Q^T * Q:
    1.000000     0.000000    -0.000000
    0.000000     1.000000    -0.000000
   -0.000000    -0.000000     1.000000

Матрица A^(0) после приведения к форме Хессенберга:
   -1.000000    -8.973818     2.910428
   -8.246211    -0.823529    -3.705882
    0.000000    -8.705882    -5.176471

Проверка формы Хессенберга: True

Матрица после QR-итераций:
  -10.571235    -0.410700    -4.923106
    2.316726     9.684269     1.523762
    0.000000    -0.000761    -6.113034

Собственные значения:
lambda1 = -6.113034
lambda2 = 9.637186
lambda3 = -10.524152

Количество QR-итераций: 19
```

1.5.5. Анализ результатов

Проверка $QR = A$ и $Q^T Q = E$ выполняется с точностью округления. После 19 QR-итераций получены собственные значения

$$\lambda_1 = -6.113034, \qquad \lambda_2 = 9.637186, \qquad \lambda_3 = -10.524152.$$

1.6. Выводы

В первой части реализованы прямые и итерационные методы численной линейной алгебры. LU-разложение позволяет одновременно решить СЛАУ, найти определитель и обратную матрицу. Для трёхдиагональной системы эффективен метод прогонки. В итерационной задаче метод Зейделя оказался быстрее метода Якоби. Методы вращений и QR-итераций дали собственные значения матриц с заданной точностью.

2. Решение нелинейных уравнений

2.1. Задание 2.1

2.1.1. Формулировка задания

Реализовать методы Ньютона и простой итерации для решения нелинейного уравнения. Найти положительный корень с заданной точностью и сравнить сходимость методов.

2.1.2. Вариант № 24

$$x^6 - 5x - 2 = 0, \qquad x \in [1, 2], \qquad \varepsilon = 10^{-6}.$$

2.1.3. Основные формулы

Для функции $f(x) = x^6 - 5x - 2$ метод Ньютона задаётся формулой

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \qquad f'(x) = 6x^5 - 5.$$

Для простой итерации уравнение переписывается в виде

$$x = \varphi(x), \qquad \varphi(x) = (5x + 2)^{1/6}, \qquad x_{k+1} = \varphi(x_k).$$

Достаточное условие сходимости простой итерации:

$$q = \max_{x \in [a,b]} |\varphi'(x)| < 1.$$

Апостериорная оценка погрешности:

$$|x^* - x_k| \le \frac{q}{1-q} |x_k - x_{k-1}|.$$

2.1.4. Текст программы

```
import math


# ============================================================
# Лабораторная работа 2.1
# Вариант 24:
#     x^6 - 5x - 2 = 0
#
# Реализованы:
# 1) метод Ньютона
# 2) метод простой итерации
#
# Алгоритм и критерии остановки записаны по методичке.
# ============================================================


# ------------------------------------------------------------
# Исходная функция и ее производные
# ------------------------------------------------------------
def f(x: float) -> float:
    """Исходная функция f(x) = x^6 - 5x - 2."""
    return x**6 - 5 * x - 2


def df(x: float) -> float:
    """Первая производная f'(x) = 6x^5 - 5."""
    return 6 * x**5 - 5


def d2f(x: float) -> float:
    """Вторая производная f''(x) = 30x^4."""
    return 30 * x**4


# ------------------------------------------------------------
# Функция phi(x) для метода простой итерации
#
```

```
# Из уравнения:
#     x^6 - 5x - 2 = 0
# получаем:
#     x^6 = 5x + 2
#     x = (5x + 2)^(1/6) = phi(x)
# ------------------------------------------------------------
def phi(x: float) -> float:
    """Функция phi(x) = (5x + 2)^(1/6)."""
    return (5 * x + 2) ** (1 / 6)


def dphi(x: float) -> float:
    """Производная phi'(x) = 5 / (6 * (5x + 2)^(5/6))."""
    return 5 / (6 * (5 * x + 2) ** (5 / 6))


# ------------------------------------------------------------
# Вспомогательная функция:
# проверка сохранения знака функции g(x) на [a, b]
#
# В методичке требуется проверять знак производных на отрезке.
# Так как в программе невозможно проверить "для всех x" буквально,
# делаем численную проверку на плотной сетке внутри отрезка.
# Это именно программная реализация проверки условия методички.
# ------------------------------------------------------------
def sign_is_constant_on_segment(g, a: float, b: float, steps: int = 10000) ->
↪  tuple[bool, int]:
    """
    Проверяет, сохраняет ли функция g(x) знак на [a, b].

    Возвращает:
        (True, sign), если знак сохраняется,
        (False, 0), если знак меняется или где-то близок к нулю.

    sign = 1  -> функция положительна на отрезке
    sign = -1 -> функция отрицательна на отрезке
    """
    h = (b - a) / steps
    eps_zero = 1e-14

    first_value = g(a)
    if abs(first_value) < eps_zero:
        return False, 0

    sign = 1 if first_value > 0 else -1

    for i in range(steps + 1):
        x = a + i * h
        value = g(x)

        if abs(value) < eps_zero:
            return False, 0

        current_sign = 1 if value > 0 else -1
        if current_sign != sign:
            return False, 0

    return True, sign
```

```
# ------------------------------------------------------------
# Проверка условий локализации корня по методичке
# ------------------------------------------------------------
def check_root_isolation(a: float, b: float) -> None:
    """
    Проверяет условия отделения корня на [a, b]:
    1) f(a) * f(b) < 0
    2) f'(x) сохраняет знак на [a, b]  -> функция строго монотонна
    """
    fa = f(a)
    fb = f(b)

    if fa * fb >= 0:
        raise ValueError(
            "На отрезке [a, b] не выполняется условие f(a) * f(b) < 0. "
            "Корень не отделен."
        )

    monotonic, sign_df = sign_is_constant_on_segment(df, a, b)
    if not monotonic:
        raise ValueError(
            "На отрезке [a, b] первая производная f'(x) не сохраняет знак. "
            "Нельзя утверждать строгую монотонность по методичке."
        )

    print("ПРОВЕРКА ОТДЕЛЕНИЯ КОРНЯ")
    print(f"Отрезок: [{a}, {b}]")
    print(f"f(a) = {fa:.10f}")
    print(f"f(b) = {fb:.10f}")
    print(f"f(a) * f(b) = {fa * fb:.10f} < 0")
    print(
        "f'(x) сохраняет знак на [a, b]: "
        + ("положительный" if sign_df > 0 else "отрицательный")
    )
    print("Следовательно, на [a, b] содержится единственный корень.\n")


# ------------------------------------------------------------
# Метод Ньютона
# ------------------------------------------------------------
def choose_initial_newton(a: float, b: float) -> float:
    """
    Выбор начального приближения по условию методички:
        f(x0) * f''(x0) > 0
    """
    left_condition = f(a) * d2f(a)
    right_condition = f(b) * d2f(b)

    if left_condition > 0:
        return a
    if right_condition > 0:
        return b

    raise ValueError(
        "Не удалось выбрать начальное приближение для метода Ньютона "
        "по условию f(x0) * f''(x0) > 0."
    )


def newton_method(a: float, b: float, eps: float) -> list[dict]:
```

```
"""
Метод Ньютона по формуле:
    x_(k+1) = x_k - f(x_k) / f'(x_k)

Остановка:
    |x_(k+1) - x_k| < eps
"""
# Проверка постоянства знаков f'(x) и f''(x) на [a, b]
const_df, sign_df = sign_is_constant_on_segment(df, a, b)
const_d2f, sign_d2f = sign_is_constant_on_segment(d2f, a, b)

if not const_df:
    raise ValueError("На [a, b] f'(x) не сохраняет знак. Условие метода
    ↪  Ньютона не выполнено.")
if not const_d2f:
    raise ValueError("На [a, b] f''(x) не сохраняет знак. Условие метода
    ↪  Ньютона не выполнено.")

xk = choose_initial_newton(a, b)

print("МЕТОД НЬЮТОНА")
print(f"f'(x) сохраняет знак на [a, b]: {'+' if sign_df > 0 else '-'}")
print(f"f''(x) сохраняет знак на [a, b]: {'+' if sign_d2f > 0 else '-'}")
print(f"Начальное приближение x0 = {xk:.10f}")
print("Останов: |x_(k+1) - x_k| < eps\n")

rows = []
k = 0

while True:
    fx = f(xk)
    dfx = df(xk)

    if abs(dfx) < 1e-14:
        raise ZeroDivisionError("f'(x_k) = 0, метод Ньютона продолжить
        ↪  нельзя.")

    correction = fx / dfx
    x_next = xk - correction
    diff = abs(x_next - xk)

    rows.append({
        "k": k,
        "xk": xk,
        "f_xk": fx,
        "df_xk": dfx,
        "corr": correction,
        "x_next": x_next,
        "diff": diff
    })

    if diff < eps:
        break

    xk = x_next
    k += 1

return rows
```

```
# ------------------------------------------------------------
# Метод простой итерации
# ------------------------------------------------------------
def check_simple_iteration_conditions(a: float, b: float) -> float:
    """
    Проверяет условия теоремы для метода простой итерации:
    1) phi(x) принадлежит [a, b] для всех x из [a, b]
    2) существует q < 1, такое что |phi'(x)| <= q на [a, b]

    Возвращает найденное q.
    """
    # Проверка phi([a, b]) □ [a, b]
    steps = 10000
    h = (b - a) / steps

    min_phi = phi(a)
    max_phi = phi(a)

    for i in range(steps + 1):
        x = a + i * h
        value = phi(x)

        if value < min_phi:
            min_phi = value
        if value > max_phi:
            max_phi = value

    if min_phi < a or max_phi > b:
        raise ValueError(
            "Не выполнено условие phi([a, b]) □ [a, b] для метода простой
            ↪  итерации."
        )

    # Проверка |phi'(x)| <= q < 1
    q = abs(dphi(a))
    for i in range(steps + 1):
        x = a + i * h
        value = abs(dphi(x))
        if value > q:
            q = value

    if q >= 1:
        raise ValueError(
            "Не выполнено условие |phi'(x)| <= q < 1 для метода простой
            ↪  итерации."
        )

    print("МЕТОД ПРОСТОЙ ИТЕРАЦИИ")
    print(f"phi([a, b]) □ [a, b]: [{min_phi:.10f}, {max_phi:.10f}] □ [{a},
    ↪  {b}]")
    print(f"q = max|phi'(x)| на [a, b] = {q:.10f} < 1")
    print("Начальное приближение по методичке берем из [a, b].")
    print("Удобно взять середину отрезка:")
    print(f"x0 = (a + b) / 2 = {(a + b) / 2:.10f}")
    print("Останов: q/(1-q) * |x_(k+1) - x_k| <= eps\n")

    return q


def simple_iteration_method(a: float, b: float, eps: float) ->
↪  tuple[list[dict], float]:
```

```
    """
    Метод простой итерации по формуле:
        x_(k+1) = phi(x_k)

    Останов:
        q/(1-q) * |x_(k+1) - x_k| <= eps
    """
    q = check_simple_iteration_conditions(a, b)
    xk = (a + b) / 2

    rows = []
    k = 0

    while True:
        x_next = phi(xk)
        diff = abs(x_next - xk)
        estimate = q / (1 - q) * diff

        rows.append({
            "k": k,
            "xk": xk,
            "phi_xk": x_next,
            "diff": diff,
            "estimate": estimate
        })

        if estimate <= eps:
            break

        xk = x_next
        k += 1

    return rows, q


# ------------------------------------------------------------
# Вспомогательный расчет "точного" значения корня
# Нужен только для анализа зависимости погрешности от числа итераций.
# ------------------------------------------------------------
def reference_root() -> float:
    """
    Находим очень точное значение корня тем же методом Ньютона,
    но с гораздо меньшей точностью.
    Это нужно только для анализа уменьшения фактической погрешности.
    """
    x = 2.0
    eps_ref = 1e-15

    while True:
        x_next = x - f(x) / df(x)
        if abs(x_next - x) < eps_ref:
            return x_next
        x = x_next


# ------------------------------------------------------------
# Печать таблиц
# ------------------------------------------------------------
def print_newton_table(rows: list[dict], x_ref: float) -> None:
    print("Таблица итераций метода Ньютона")
```

```
    print(
        f"{'k':>3} "
        f"{'x_k':>16} "
        f"{'f(x_k)':>16} "
        f"{'f (x_k)':>16} "
        f"{'f/df':>16} "
        f"{'x_(k+1)':>16} "
        f"{'|x_(k+1)-x_k|':>18} "
        f"{'|x_k-x*|':>16}"
    )

    for row in rows:
        error = abs(row["xk"] - x_ref)
        print(
            f"{row['k']:3d} "
            f"{row['xk']:16.10f} "
            f"{row['f_xk']:16.10f} "
            f"{row['df_xk']:16.10f} "
            f"{row['corr']:16.10f} "
            f"{row['x_next']:16.10f} "
            f"{row['diff']:18.10e} "
            f"{error:16.10e}"
        )
    print()


def print_simple_iteration_table(rows: list[dict], x_ref: float) -> None:
    print("Таблица итераций метода простой итерации")
    print(
        f"{'k':>3} "
        f"{'x_k':>16} "
        f"{'phi(x_k)':>16} "
        f"{'|x_(k+1)-x_k|':>18} "
        f"{'Оценка погр.':>16} "
        f"{'|x_k-x*|':>16}"
    )

    for row in rows:
        error = abs(row["xk"] - x_ref)
        print(
            f"{row['k']:3d} "
            f"{row['xk']:16.10f} "
            f"{row['phi_xk']:16.10f} "
            f"{row['diff']:18.10e} "
            f"{row['estimate']:16.10e} "
            f"{error:16.10e}"
        )
    print()


# ------------------------------------------------------------
# Главная функция
# ------------------------------------------------------------
def main():
    # Входные данные: требуемая точность
    eps = float(input("Введите точность eps: "))

    # Положительный корень по графическому отделению берем на отрезке [1, 2]
    a = 1.0
    b = 2.0
```

```
    print("\nУРАВНЕНИЕ: x^6 - 5x - 2 = 0")
    print("Ищем положительный корень.\n")

    # 1. Проверка отделения корня
    check_root_isolation(a, b)

    # 2. Метод Ньютона
    newton_rows = newton_method(a, b, eps)

    # 3. Метод простой итерации
    simple_rows, q = simple_iteration_method(a, b, eps)

    # 4. Эталонный корень для анализа погрешности
    x_ref = reference_root()

    # 5. Вывод результатов
    print(f"Эталонное значение корня для анализа погрешности: x* =
    ↪  {x_ref:.15f}\n")

    print_newton_table(newton_rows, x_ref)
    print_simple_iteration_table(simple_rows, x_ref)

    # 6. Итоговые приближения
    x_newton = newton_rows[-1]["x_next"]
    x_simple = simple_rows[-1]["phi_xk"]

    print("ИТОГ")
    print(f"Метод Ньютона:           x ≈ {x_newton:.10f}, число итераций =
    ↪  {len(newton_rows)}")
    print(f"Метод простой итерации:  x ≈ {x_simple:.10f}, число итераций =
    ↪  {len(simple_rows)}")

    # 7. Краткий анализ зависимости погрешности от числа итераций
    print("\nАНАЛИЗ ПОГРЕШНОСТИ")
    print("По таблицам видно, что с ростом числа итераций погрешность
    ↪  уменьшается.")
    print("Метод Ньютона сходится быстрее, чем метод простой итерации.")
    print("У метода простой итерации сходимость медленнее, так как
    ↪  используется итерационная схема")
    print("x_(k+1) = phi(x_k) и теоретическая оценка погрешности через
    ↪  коэффициент q.")


if __name__ == "__main__":
    main()
```

Вывод программы:

```
Введите точность eps:
УРАВНЕНИЕ: x^6 - 5x - 2 = 0
Ищем положительный корень.

ПРОВЕРКА ОТДЕЛЕНИЯ КОРНЯ
Отрезок: [1.0, 2.0]
f(a) = -6.0000000000
f(b) = 52.0000000000
f(a) * f(b) = -312.0000000000 < 0
f'(x) сохраняет знак на [a, b]: положительный
```

```
Следовательно, на [a, b] содержится единственный корень.

МЕТОД НЬЮТОНА
f'(x) сохраняет знак на [a, b]: +
f''(x) сохраняет знак на [a, b]: +
Начальное приближение x0 = 2.0000000000
Останов: |x_(k+1) - x_k| < eps

МЕТОД ПРОСТОЙ ИТЕРАЦИИ
phi([a, b]) □ [a, b]: [1.3830875543, 1.5130857494] □ [1.0, 2.0]
q = max|phi'(x)| на [a, b] = 0.1646532803 < 1
Начальное приближение по методичке берем из [a, b].
Удобно взять середину отрезка:
x0 = (a + b) / 2 = 1.5000000000
Останов: q/(1-q) * |x_(k+1) - x_k| <= eps

Эталонное значение корня для анализа погрешности: x* = 1.448678056364867

Таблица итераций метода Ньютона
  k              x_k           f(x_k)          f (x_k)             f/df
   ↪  x_(k+1)       |x_(k+1)-x_k|          |x_k-x*|
  0     2.0000000000    52.0000000000   187.0000000000     0.2780748663
   ↪  1.7219251337   2.7807486631e-01 5.5132194364e-01
  1     1.7219251337    15.4570465571    85.8285907983     0.1800920464
   ↪  1.5418330873   1.8009204641e-01 2.7324707732e-01
  2     1.5418330873     3.7254168464    47.2802982772     0.0787942755
   ↪  1.4630388118   7.8794275462e-02 9.3155030918e-02
  3     1.4630388118     0.4917812986    35.2189276667     0.0139635512
   ↪  1.4490752607   1.3963551169e-02 1.4360755456e-02
  4     1.4490752607     0.0132307418    33.3359262138     0.0003968914
   ↪  1.4486783693   3.9689138252e-04 3.9720428689e-04
  5     1.4486783693     0.0000104145    33.2834552802     0.0000003129
   ↪  1.4486780564   3.1290417923e-07 3.1290437352e-07

Таблица итераций метода простой итерации
  k              x_k         phi(x_k)      |x_(k+1)-x_k|     Оценка погр.
   ↪   |x_k-x*|
  0     1.5000000000     1.4553047077   4.4695292343e-02 8.8097867905e-03
   ↪  5.1321943635e-02
  1     1.4553047077     1.4495422394   5.7624682212e-03 1.1358269239e-03
   ↪  6.6266512917e-03
  2     1.4495422394     1.4487909009   7.5133853699e-04 1.4809461962e-04
   ↪  8.6418307049e-04
  3     1.4487909009     1.4486927940   9.8106863588e-05 1.9337619368e-05
   ↪  1.1284453350e-04
  4     1.4486927940     1.4486799812   1.2812864962e-05 2.5255144909e-06
   ↪  1.4737669916e-05
  5     1.4486799812     1.4486783078   1.6734161776e-06 3.2984323323e-07
   ↪  1.9248049543e-06

ИТОГ
Метод Ньютона:           x ≈ 1.4486780564, число итераций = 6
Метод простой итерации:  x ≈ 1.4486783078, число итераций = 6

АНАЛИЗ ПОГРЕШНОСТИ
По таблицам видно, что с ростом числа итераций погрешность уменьшается.
Метод Ньютона сходится быстрее, чем метод простой итерации.
У метода простой итерации сходимость медленнее, так как используется
 ↪  итерационная схема
x_(k+1) = phi(x_k) и теоретическая оценка погрешности через коэффициент q.
```

2.1.5. Анализ сходимости

На отрезке $[1, 2]$ функция меняет знак, а её первая производная сохраняет знак, поэтому положительный корень единственен. Для метода простой итерации получено $q = 0.1646532803 < 1$. При точности 10^{-6} оба метода выполнили 6 итераций; метод Ньютона дал значение $x = 1.4486780564$, практически совпадающее с контрольным корнем.

2.2. Задание 2.2

2.2.1. Формулировка задания

Реализовать методы Ньютона и простой итерации для решения системы нелинейных уравнений. Начальное приближение определить графически и исследовать зависимость погрешности от числа итераций.

2.2.2. Вариант № 24

$$\begin{cases} 3x_1^2 - x_1 + x_2^2 - 1 = 0, \\ x_2 - \tan x_1 = 0, \end{cases} \qquad \varepsilon = 10^{-6}.$$

2.2.3. Основные формулы

Для системы $F(x) = 0$ итерация Ньютона записывается как

$$J(x^{(k)})\Delta x^{(k)} = -F(x^{(k)}), \qquad x^{(k+1)} = x^{(k)} + \Delta x^{(k)},$$

где J --- матрица Якоби системы. Для метода простой итерации используется преобразование $x = \varphi(x)$ и процесс

$$x^{(k+1)} = \varphi(x^{(k)}).$$

Достаточное условие сходимости в выбранной области G:

$$\max_{x \in G} \|\varphi'(x)\|_\infty = q < 1.$$

2.2.4. Графическое определение начального приближения

По пересечению кривых выбрана область

$$G: \quad x_1 \in (0.58, 0.64), \qquad x_2 \in (0.67, 0.73),$$

и начальное приближение $x^{(0)} = (0.6,\ 0.7)^T$.

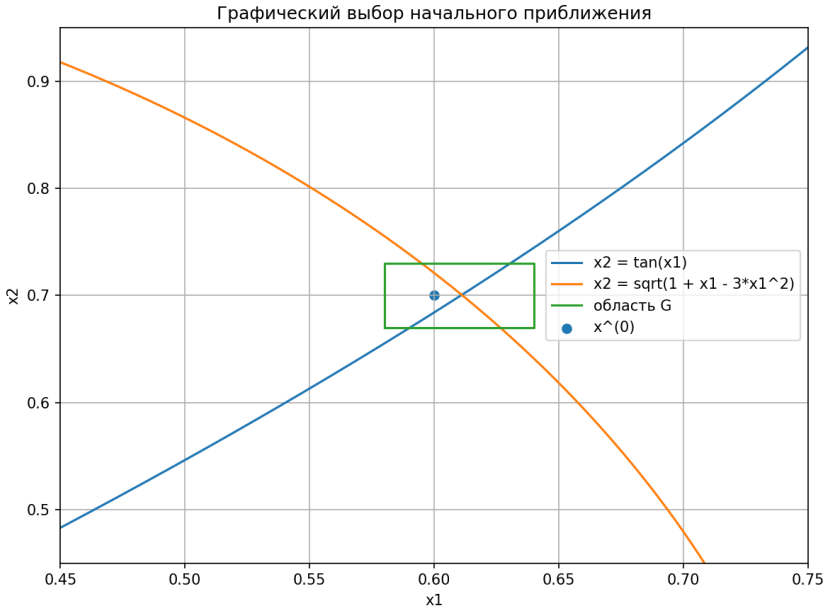


Рис. 1: Графический выбор начального приближения

2.2.5. Текст программы

```
import math
import matplotlib.pyplot as plt

# ============================================================
# Лабораторная работа 2.2, вариант 24
# Система:000
#   3*x1^2 - x1 + x2^2 - 1 = 0
#   x2 - tan(x1) = 0
#
# В коде соблюдены шаги из методички:
# 1) начальное приближение выбирается графически;
# 2) метод Ньютона реализован по формулам (2.13)-(2.20);
# 3) метод простой итерации реализован в форме x = phi(x);
# 4) для простой итерации проверяются q < 1 и попадание всех итераций в область
 ↪  G;
# 5) критерии остановки соответствуют (2.18) и (2.26).
# ============================================================

DEFAULT_EPS = 1e-6
MAX_ITER = 100
GRID_STEPS_Q = 200
GRID_STEPS_G = 80

# По графику берем область положительного решения и начальное приближение
G_X1 = (0.58, 0.64)
G_X2 = (0.67, 0.73)
X0 = (0.60, 0.70)


# ----------------------------
# Исходная система и Якобиан
# ----------------------------
def f1(x1, x2):
    return 3.0 * x1 * x1 - x1 + x2 * x2 - 1.0
```

```
def f2(x1, x2):
    return x2 - math.tan(x1)


def jacobian(x1, x2):
    # J = [[df1/dx1, df1/dx2], [df2/dx1, df2/dx2]]
    return (
        (6.0 * x1 - 1.0, 2.0 * x2),
        (-(1.0 / (math.cos(x1) ** 2)), 1.0),
    )


# ----------------------------
# Вспомогательные функции
# ----------------------------
def det2(m):
    return m[0][0] * m[1][1] - m[0][1] * m[1][0]



def norm_inf_vec(v):
    return max(abs(v[0]), abs(v[1]))



def in_G(x1, x2):
    return G_X1[0] <= x1 <= G_X1[1] and G_X2[0] <= x2 <= G_X2[1]


# ------------------------------------------------------------
# Шаг 1. Графическое определение начального приближения
# ------------------------------------------------------------
def plot_system():
    x_values = []
    tan_values = []
    sqrt_values = []

    left = -0.6
    right = 0.8
    step = 0.001
    x = left
    while x <= right:
        x_values.append(x)
        tan_values.append(math.tan(x))

        rad = 1.0 + x - 3.0 * x * x
        if rad >= 0.0:
            sqrt_values.append(math.sqrt(rad))
        else:
            sqrt_values.append(float('nan'))
        x += step

    plt.figure(figsize=(8, 6))
    plt.plot(x_values, tan_values, label='x2 = tan(x1)')
    plt.plot(x_values, sqrt_values, label='x2 = sqrt(1 + x1 - 3*x1^2)')

    # Показываем прямоугольник G и начальную точку
    gx1 = [G_X1[0], G_X1[1], G_X1[1], G_X1[0], G_X1[0]]
    gx2 = [G_X2[0], G_X2[0], G_X2[1], G_X2[1], G_X2[0]]
```

```
    plt.plot(gx1, gx2, label='область G')
    plt.scatter([X0[0]], [X0[1]], label='x^(0)')

    plt.xlim(0.45, 0.75)
    plt.ylim(0.45, 0.95)
    plt.xlabel('x1')
    plt.ylabel('x2')
    plt.title('Графический выбор начального приближения')
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig('lab2_2_graph.png', dpi=150)
    plt.close()


# ---------------------------------------------------------------------
# Шаг 2. Метод Ньютона по формулам (2.13)-(2.20) для системы 2x2
# ---------------------------------------------------------------------
# Для двух уравнений используем форму из примера методички:
# x1^(k+1) = x1^(k) - det(A1) / det(J)
# x2^(k+1) = x2^(k) - det(A2) / det(J)

def newton_method(eps, max_iter=MAX_ITER):
    x1, x2 = X0
    history = []

    for k in range(max_iter):
        cur_f1 = f1(x1, x2)
        cur_f2 = f2(x1, x2)

        J = jacobian(x1, x2)
        det_j = det2(J)
        if abs(det_j) < 1e-14:
            raise ValueError(
                f'Метод Ньютона остановлен: det(J) = 0 на итерации k = {k}.'
            )

        # Матрицы A1 и A2 из формулы (2.20)
        A1 = (
            (cur_f1, J[0][1]),
            (cur_f2, J[1][1]),
        )
        A2 = (
            (J[0][0], cur_f1),
            (J[1][0], cur_f2),
        )

        det_a1 = det2(A1)
        det_a2 = det2(A2)

        x1_new = x1 - det_a1 / det_j
        x2_new = x2 - det_a2 / det_j

        diff = norm_inf_vec((x1_new - x1, x2_new - x2))

        history.append({
            'k': k,
            'x1': x1,
            'x2': x2,
            'f1': cur_f1,
```

```
            'f2': cur_f2,
            'detJ': det_j,
            'detA1': det_a1,
            'detA2': det_a2,
            'delta': diff,
            'x1_next': x1_new,
            'x2_next': x2_new,
        })

        x1, x2 = x1_new, x2_new

        # Критерий остановки (2.18)
        if diff <= eps:
            return (x1, x2), history

    raise ValueError('Метод Ньютона не сошелся за заданное число итераций.')


# ------------------------------------------------------------------------------
↪  ----------
# Шаг 3. Метод простой итерации: приводим систему к виду x = phi(x)
# ------------------------------------------------------------------------------
↪  ----------
# Берем эквивалентную систему x = phi(x) вида
#   x^(k+1) = x^(k) - C * f(x^(k)),
# где C = J(x^(0))^{-1} - постоянная матрица.
# Это все еще форма (2.21)-(2.24) из методички.

def inverse_2x2(m):
    d = det2(m)
    if abs(d) < 1e-14:
        raise ValueError('Невозможно обратить матрицу 2x2: det = 0.')
    return (
        (m[1][1] / d, -m[0][1] / d),
        (-m[1][0] / d, m[0][0] / d),
    )


J0 = jacobian(X0[0], X0[1])
C = inverse_2x2(J0)


def phi(x1, x2):
    cur_f1 = f1(x1, x2)
    cur_f2 = f2(x1, x2)

    p1 = x1 - (C[0][0] * cur_f1 + C[0][1] * cur_f2)
    p2 = x2 - (C[1][0] * cur_f1 + C[1][1] * cur_f2)
    return p1, p2



def dphi(x1, x2):
    # Производная phi(x) = I - C*J(x)
    sec2 = 1.0 / (math.cos(x1) ** 2)
    c11, c12 = C[0]
    c21, c22 = C[1]

    return (
        (
```

```
            1.0 - c11 * (6.0 * x1 - 1.0) + c12 * sec2,
            -(2.0 * c11 * x2 + c12),
        ),
        (
            -c21 * (6.0 * x1 - 1.0) + c22 * sec2,
            1.0 - 2.0 * c21 * x2 - c22,
        ),
    )


def compute_q_on_G(grid_steps=GRID_STEPS_Q):
    # Проверка условия (2.25) по формуле (2.27):
    # q = max_{x in G} max_i sum_j |d(phi_i)/d(x_j)|
    q = -1.0
    q_point = None

    for i in range(grid_steps + 1):
        x1 = G_X1[0] + (G_X1[1] - G_X1[0]) * i / grid_steps
        for j in range(grid_steps + 1):
            x2 = G_X2[0] + (G_X2[1] - G_X2[0]) * j / grid_steps
            d = dphi(x1, x2)
            row1 = abs(d[0][0]) + abs(d[0][1])
            row2 = abs(d[1][0]) + abs(d[1][1])
            current = max(row1, row2)
            if current > q:
                q = current
                q_point = (x1, x2)

    return q, q_point


def check_phi_maps_G_into_G(grid_steps=GRID_STEPS_G):
    # Дополнительная численная проверка: phi(G)  G
    for i in range(grid_steps + 1):
        x1 = G_X1[0] + (G_X1[1] - G_X1[0]) * i / grid_steps
        for j in range(grid_steps + 1):
            x2 = G_X2[0] + (G_X2[1] - G_X2[0]) * j / grid_steps
            p1, p2 = phi(x1, x2)
            if not in_G(p1, p2):
                return False, (x1, x2), (p1, p2)
    return True, None, None


def simple_iteration_method(eps, max_iter=MAX_ITER):
    q, q_point = compute_q_on_G()
    if q >= 1.0:
        raise ValueError(
            f'Метод простой итерации неприменим: q = {q:.10f} >= 1.'
        )

    maps_ok, src_point, mapped_point = check_phi_maps_G_into_G()
    if not maps_ok:
        raise ValueError(
            'Численная проверка показала, что phi(G) не содержится в G: '
            f'phi({src_point}) = {mapped_point}.'
        )
```

```
    x1, x2 = X0
    history = []

    for k in range(max_iter):
        x1_new, x2_new = phi(x1, x2)

        if not in_G(x1_new, x2_new):
            raise ValueError(
              f'Итерация k = {k + 1} вышла из области G: ({x1_new}, {x2_new}).'
            )

        diff = norm_inf_vec((x1_new - x1, x2_new - x2))
        err_est = q / (1.0 - q) * diff  # критерий из (2.26)

        history.append({
            'k': k,
            'x1': x1,
            'x2': x2,
            'phi1': x1_new,
            'phi2': x2_new,
            'delta': diff,
            'err_est': err_est,
            'q': q,
        })

        x1, x2 = x1_new, x2_new

        # Критерий остановки (2.26)
        if err_est <= eps:
            return (x1, x2), history, q, q_point

    raise ValueError('Метод простой итерации не сошелся за заданное число
    ↪  итераций.')


# ------------------------------------------------------------
# Вывод таблиц и анализ погрешности по числу итераций
# ------------------------------------------------------------
def print_newton_table(history):
    print('\nМЕТОД НЬЮТОНА')
    print('-' * 120)
    print(
        f"{'k':>2} | {'x1^(k)':>12} | {'x2^(k)':>12} | {'f1':>12} | {'f2':>12}
        ↪  | "
        f"{'detJ':>12} | {'detA1':>12} | {'detA2':>12} |
        ↪  {'||x^(k+1)-x^(k)||∞':>18}"
    )
    print('-' * 120)
    for row in history:
        print(
            f"{row['k']:>2} | {row['x1']:>12.8f} | {row['x2']:>12.8f} | "
           f"{row['f1']:>12.8f} | {row['f2']:>12.8f} | {row['detJ']:>12.8f} | "
            f"{row['detA1']:>12.8f} | {row['detA2']:>12.8f} |
            ↪  {row['delta']:>18.10f}"
        )



def print_simple_iteration_table(history, q, q_point):
    print('\nМЕТОД ПРОСТОЙ ИТЕРАЦИИ')
```

```
    print('-' * 120)
    print(f'q = {q:.10f}, точка максимума на проверочной сетке:
    ↪  ({q_point[0]:.6f}, {q_point[1]:.6f})')
    print(
        f"{'k':>2} | {'x1^(k)':>12} | {'x2^(k)':>12} | {'phi1':>12} |
        ↪  {'phi2':>12} | "
        f"{'||x^(k+1)-x^(k)||∞':>18} | {'оценка ошибки':>16}"
    )
    print('-' * 120)
    for row in history:
        print(
            f"{row['k']:>2} | {row['x1']:>12.8f} | {row['x2']:>12.8f} | "
            f"{row['phi1']:>12.8f} | {row['phi2']:>12.8f} | "
            f"{row['delta']:>18.10f} | {row['err_est']:>16.10f}"
        )



def newton_reference_solution():
    # Нужна только для анализа зависимости фактической ошибки от номера
    ↪  итерации
    sol, _ = newton_method(eps=1e-14, max_iter=200)
    return sol



def plot_error_analysis(newton_history, simple_history, reference_solution):
    newton_errors = []
    newton_k = []
    for row in newton_history:
        err = norm_inf_vec((row['x1_next'] - reference_solution[0],
        ↪  row['x2_next'] - reference_solution[1]))
        newton_errors.append(err)
        newton_k.append(row['k'] + 1)

    simple_errors = []
    simple_k = []
    for row in simple_history:
        err = norm_inf_vec((row['phi1'] - reference_solution[0], row['phi2'] -
        ↪  reference_solution[1]))
        simple_errors.append(err)
        simple_k.append(row['k'] + 1)

    plt.figure(figsize=(8, 6))
    plt.semilogy(newton_k, newton_errors, marker='o', label='Ньютон:
    ↪  фактическая ошибка')
    plt.semilogy(simple_k, simple_errors, marker='s', label='Простая итерация:
    ↪  фактическая ошибка')
    plt.xlabel('Номер итерации k')
    plt.ylabel('||x^(k) - x*||∞')
    plt.title('Зависимость погрешности от количества итераций')
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig('lab2_2_errors.png', dpi=150)
    plt.close()


# ------------------------------------------------------------
# Главная часть
```

```
# ------------------------------------------------------------
def main():
    print('ЛР 2.2, вариант 24')
    print('Система:')
    print('  3*x1^2 - x1 + x2^2 - 1 = 0')
    print('  x2 - tan(x1) = 0')
    try:
        raw_eps = input('Введите точность eps (например, 1e-6). Если оставить
        ↪  пусто, будет 1e-6: ').strip()
        eps = DEFAULT_EPS if raw_eps == '' else float(raw_eps)
    except ValueError:
        raise ValueError('Точность eps должна быть числом.')

    if eps <= 0:
        raise ValueError('Точность eps должна быть положительной.')

    print(f'Точность eps = {eps}')
    print(f'Область G: x1 in {G_X1}, x2 in {G_X2}')
    print(f'Начальное приближение x^(0) = {X0}')

    # 1. График для выбора начального приближения
    plot_system()

    # 2. Метод Ньютона
    newton_solution, newton_history = newton_method(eps=eps)
    print_newton_table(newton_history)
    print(f"\nРезультат метода Ньютона: x1 = {newton_solution[0]:.10f}, x2 =
    ↪  {newton_solution[1]:.10f}")

    # 3. Метод простой итерации
    simple_solution, simple_history, q, q_point =
    ↪  simple_iteration_method(eps=eps)
    print_simple_iteration_table(simple_history, q, q_point)
    print(f"\nРезультат метода простой итерации: x1 =
    ↪  {simple_solution[0]:.10f}, x2 = {simple_solution[1]:.10f}")

    # 4. Анализ погрешности по числу итераций
    ref = newton_reference_solution()
    plot_error_analysis(newton_history, simple_history, ref)

    print('\nСправочно: решение повышенной точности для анализа графика
    ↪  ошибок:')
    print(f'x* = ({ref[0]:.14f}, {ref[1]:.14f})')


if __name__ == '__main__':
    main()
```

Вывод программы:

```
ЛР 2.2, вариант 24
Система:
  3*x1^2 - x1 + x2^2 - 1 = 0
  x2 - tan(x1) = 0
Введите точность eps (например, 1e-6). Если оставить пусто, будет 1e-6:
↪  Точность eps = 1e-06
Область G: x1 in (0.58, 0.64), x2 in (0.67, 0.73)
Начальное приближение x^(0) = (0.6, 0.7)
```

```
МЕТОД НЬЮТОНА
--------------------------------------------------------------------------------
 ↪  ----------------------------------------
 k |       x1^(k) |       x2^(k) |           f1 |           f2 |         detJ |
  ↪       detA1 |        detA2 | ||x^(k+1)-x^(k)||∞
--------------------------------------------------------------------------------
 ↪  ----------------------------------------
 0 |   0.60000000 |   0.70000000 |  -0.03000000 |   0.01586319 |   4.65526044 |
  ↪   -0.05220847 |  -0.00279700 |       0.0112149404
 1 |   0.61121494 |   0.70060083 |   0.00037769 |  -0.00012800 |   4.75651057 |
  ↪    0.00055704 |   0.00022173 |       0.0001171106
 2 |   0.61109783 |   0.70055421 |   0.00000004 |  -0.00000001 |   4.75532609 |
  ↪    0.00000006 |   0.00000003 |       0.0000000133

Результат метода Ньютона: x1 = 0.6110978165, x2 = 0.7005542039

МЕТОД ПРОСТОЙ ИТЕРАЦИИ
--------------------------------------------------------------------------------
 ↪  ----------------------------------------
q = 0.0903973619, точка максимума на проверочной сетке: (0.640000, 0.730000)
 k |       x1^(k) |       x2^(k) |         phi1 |         phi2 |
  ↪  ||x^(k+1)-x^(k)||∞ |    оценка ошибки
--------------------------------------------------------------------------------
 ↪  ----------------------------------------
 0 |   0.60000000 |   0.70000000 |   0.61121494 |   0.70060083 |
  ↪  0.0112149404 |     0.0011145537
 1 |   0.61121494 |   0.70060083 |   0.61109532 |   0.70055321 |
  ↪  0.0001196247 |     0.0000118884
 2 |   0.61109532 |   0.70055321 |   0.61109787 |   0.70055422 |
  ↪  0.0000025538 |     0.0000002538

Результат метода простой итерации: x1 = 0.6110978696, x2 = 0.7005542250

Справочно: решение повышенной точности для анализа графика ошибок:
x* = (0.61109781647175, 0.70055420388584)
```

2.2.6. Анализ сходимости

Для метода простой итерации программа вычислила $q = 0.0903973619 < 1$, следовательно, процесс сходится в области G. Метод Ньютона дал решение

$$x_1 = 0.6110978165, \qquad x_2 = 0.7005542039,$$

а метод простой итерации ---

$$x_1 = 0.6110978696, \qquad x_2 = 0.7005542250.$$

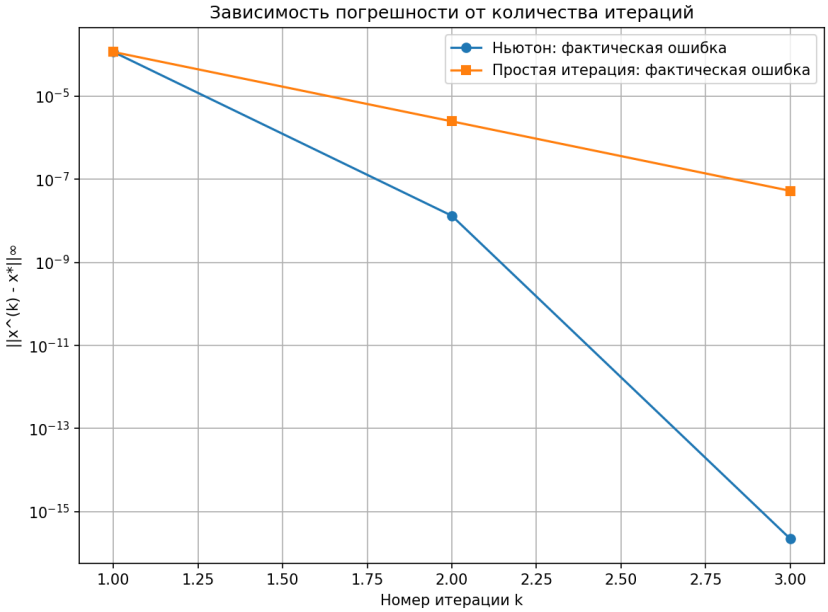


Рис. 2: Зависимость погрешности от количества итераций

2.3. Выводы

Для скалярного уравнения и системы нелинейных уравнений реализованы два итерационных подхода. Условия сходимости проверены численно на выбранных отрезке и области. Полученные решения методов Ньютона и простой итерации согласуются в пределах требуемой точности.

3. Интерполяция и аппроксимация

3.1. Задание 3.1

3.1.1. Формулировка задания

Построить интерполяционные многочлены Лагранжа и Ньютона, вычислить их значения в заданной точке и оценить абсолютную погрешность.

3.1.2. Вариант № 24

Задана функция

$$f(x) = \frac{1}{x^2}, \qquad X^* = 0.8,$$

и два набора узлов:

$$\text{а) } (0.1,\ 0.5,\ 0.9,\ 1.3), \qquad \text{б) } (0.1,\ 0.5,\ 1.1,\ 1.3).$$

3.1.3. Основные формулы

Интерполяционный многочлен Лагранжа:

$$L_n(x) = \sum_{i=0}^{n} f(x_i) \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}.$$

Интерполяционный многочлен Ньютона:

$$P_n(x) = f[x_0] + \sum_{i=1}^{n} f[x_0, \ldots, x_i] \prod_{j=0}^{i-1} (x - x_j).$$

3.1.4. Текст программы

```
# Построение интерполяционных многочленов Лагранжа и Ньютона

# fabs используется для вычисления абсолютной погрешности:
# |точное значение - приближённое значение|
from math import fabs

# 1. Исходная функция по варианту
def f(x: float) -> float:
    # По варианту 24 функция имеет вид y = 1 / x^2
    # Именно её значения мы считаем в узлах интерполяции и в точке X*
    return 1.0 / (x * x)

# 2. Проверки входных данных
def check_nodes(x_nodes):
    # Для интерполяции нужно хотя бы две точки.
    # Если точек меньше, построить интерполяционный многочлен нельзя.
    if len(x_nodes) < 2:
        raise ValueError("Ошибка: для интерполяции нужно минимум 2 узла.")

    # Проверка несовпадения узлов
    # Если какие-то x_i совпадают, то в формулах Лагранжа и Ньютона
    # появится деление на ноль, поэтому такие данные недопустимы
    for i in range(len(x_nodes)):
        for j in range(i + 1, len(x_nodes)):
            if x_nodes[i] == x_nodes[j]:
                raise ValueError("Ошибка: интерполяционные узлы должны быть
                ↪  различными.")

# 3. Формирование таблицы значений функции
def make_table(x_nodes):
    # Здесь по каждому узлу x_i вычисляется значение функции y_i = f(x_i).
    # По сути, мы получаем таблично заданную функцию:
    # (x_0, y_0), (x_1, y_1), ..., (x_n, y_n)
    y_nodes = []
    for x in x_nodes:
        y_nodes.append(f(x))
    return y_nodes

# 4. Многочлен Лагранжа
#    Формула вида:
#    L_n(x) = sum( f(x_i) * l_i(x) )
def lagrange_value(x_nodes, y_nodes, x_star):
    # n = количество узлов
    n = len(x_nodes)

    # В result постепенно будет накапливаться значение многочлена Лагранжа
    # в точке x_star
    result = 0.0

    # Внешний цикл перебирает каждое слагаемое суммы Лагранжа
    for i in range(n):
        # numerator  = произведение (x* - x_j), j != i
        # denominator = произведение (x_i - x_j), j != i
```

```
        # Их отношение даёт базисный многочлен l_i(x*)
        numerator = 1.0
        denominator = 1.0

        # Строим произведения для i-го базисного многочлена
        for j in range(n):
            if i != j:
                numerator *= (x_star - x_nodes[j])
                denominator *= (x_nodes[i] - x_nodes[j])

        # Добавляем очередное слагаемое:
        # y_i * l_i(x*)
        result += y_nodes[i] * numerator / denominator

    # После окончания цикла получаем итоговое значение L_n(x*)
    return result

# Красивый вывод  === === === === === === === === === === === === ===
def lagrange_coefficients_for_print(x_nodes, y_nodes):
    # Эта функция нужна не для вычисления значения в точке,
    # а для красивой записи самого многочлена Лагранжа в виде строки.

    # Здесь считаются коэффициенты вида:
    # f(x_i) / w'(x_i)
    # как в примере из методички.
    n = len(x_nodes)
    coeffs = []

    # Для каждого узла x_i считаем произведение
    # (x_i - x_j), j != i
    for i in range(n):
        w_derivative = 1.0
        for j in range(n):
            if i != j:
                w_derivative *= (x_nodes[i] - x_nodes[j])

        # Сохраняем коэффициент перед соответствующим произведением
        coeffs.append(y_nodes[i] / w_derivative)

    return coeffs

def lagrange_polynomial_string(x_nodes, y_nodes):
    # Получаем коэффициенты для записи многочлена
    coeffs = lagrange_coefficients_for_print(x_nodes, y_nodes)
    parts = []

    # Формируем текст каждого слагаемого многочлена Лагранжа
    for i in range(len(x_nodes)):
        term = f"{coeffs[i]:+.10f}"

        # К коэффициенту дописываем множители (x - x_j), j != i
        for j in range(len(x_nodes)):
            if i != j:
                term += f" * (x - {x_nodes[j]})"

        parts.append(term)

    # Склеиваем все слагаемые в одну строку
    return " ".join(parts)
# === === === === === === === === === === === === === === === === ===
```

```
# 5. Разделённые разности для многочлена Ньютона
def divided_differences_table(x_nodes, y_nodes):
    # n = число узлов
    n = len(x_nodes)

    # Создаём квадратную таблицу n x n, заполненную нулями.
    # В ней будут храниться разделённые разности:
    # 0-го, 1-го, 2-го, ... порядка
    table = [[0.0 for _ in range(n)] for _ in range(n)]

    # Нулевой порядок разделённых разностей —
    # это просто сами значения функции в узлах:
    # f[x_i] = y_i
    for i in range(n):
        table[i][0] = y_nodes[i]

    # Далее строим разделённые разности более высоких порядков.
    #
    # order = 1  -> первый порядок
    # order = 2  -> второй порядок
    # и так далее
    for order in range(1, n):
        for i in range(n - order):
            # Знаменатель формулы разделённой разности:
            # x_{i+order} - x_i
            denominator = x_nodes[i + order] - x_nodes[i]

            # Дополнительная защита от деления на ноль
            if denominator == 0:
                raise ValueError("Ошибка: деление на ноль при вычислении
                ↪  разделённых разностей.")

            # Формула разделённой разности:
            # f[x_i, ..., x_{i+order}] =
            # ( f[x_{i+1}, ..., x_{i+order}] - f[x_i, ..., x_{i+order-1}] )
            # / (x_{i+order} - x_i)
            table[i][order] = (table[i + 1][order - 1] - table[i][order - 1]) /
            ↪  denominator

    # Возвращаем готовую таблицу разделённых разностей
    return table


# 6. Многочлен Ньютона
#    P_n(x) = f[x0] + f[x0,x1](x-x0) + ...
def newton_value(x_nodes, diff_table, x_star):
    # n = число узлов
    n = len(x_nodes)

    # Первый член многочлена Ньютона — это f[x0]
    result = diff_table[0][0]

    # В этой переменной постепенно будет накапливаться произведение:
    # (x - x0), (x - x0)(x - x1), ...
    product = 1.0

    # Добавляем остальные члены многочлена Ньютона
    for order in range(1, n):
        # На каждом шаге домножаем ещё на один множитель
```

```
            product *= (x_star - x_nodes[order - 1])

            # Прибавляем очередной член:
            # f[x0, ..., x_order] * (x-x0)...(x-x_{order-1})
            result += diff_table[0][order] * product

    # Итог — значение P_n(x*) в точке x_star
    return result


def newton_polynomial_string(x_nodes, diff_table):
    # Начинаем запись многочлена Ньютона с первого члена f[x0]
    parts = [f"{diff_table[0][0]:.10f}"]

    # Формируем остальные слагаемые
    for order in range(1, len(x_nodes)):
        # Коэффициент — это разделённая разность нужного порядка
        term = f"{diff_table[0][order]:+.10f}"

        # Добавляем множители (x - x0)(x - x1)... по формуле Ньютона
        for j in range(order):
            term += f" * (x - {x_nodes[j]})"

        parts.append(term)

    # Возвращаем строковую запись многочлена
    return " ".join(parts)


# 7. Печать таблицы разделённых разностей
def print_divided_differences_table(x_nodes, diff_table):
    # Количество узлов
    n = len(x_nodes)

    print("Таблица разделённых разностей:")

    # Формируем заголовок таблицы.
    # Первый столбец — номер i,
    # второй — x_i,
    # третий — f[x_i],
    # далее столбцы с разделёнными разностями higher order.
    header = ["i", "x_i", "f[x_i]"]
    for order in range(1, n):
        if order == 1:
            header.append("f[x_i, x_{i+1}]")
        elif order == 2:
            header.append("f[x_i, x_{i+1}, x_{i+2}]")
        else:
            header.append("f[x_i, ..., x_{i+" + str(order) + "}]")

    print("-" * 120)
    print("{:<5} {:<12} {:<18} {:<22} {:<24} {:<24}".format(*header))
    print("-" * 120)

    # Построчно печатаем таблицу
    for i in range(n):
        # Начало строки: номер узла, сам узел и значение функции
        row = [str(i), f"{x_nodes[i]:.10f}", f"{diff_table[i][0]:.10f}"]

        # Добавляем разделённые разности.
```

```
            # Не все элементы в строке существуют, поэтому для недостающих
            # ячеек позже добавляем пустые строки.
            for order in range(1, n):
                if i < n - order:
                    row.append(f"{diff_table[i][order]:.10f}")
                else:
                    row.append("")

            # Чтобы форматирование print не ломалось, добиваем строку
            # до нужного количества столбцов
            while len(row) < 6:
                row.append("")

            print("{:<5} {:<12} {:<18} {:<22} {:<24} {:<24}".format(*row))

        print("-" * 120)


# 8. Решение одного пункта варианта
def solve_case(case_name, x_nodes, x_star):
    # Красивый заголовок для отдельного случая:
    # а) или б)
    print("=" * 120)
    print(f"СЛУЧАЙ {case_name}")
    print("=" * 120)

    # Шаг 1. Проверка узлов
    # Проверяем, что узлы корректны:
    # их достаточно и они не совпадают
    check_nodes(x_nodes)

    # Шаг 2. Вычисление значений функции в узлах
    # Получаем таблицу y_i = f(x_i)
    y_nodes = make_table(x_nodes)

    print("Таблица значений функции:")
    print("-" * 50)
    print("{:<5} {:<15} {:<20}".format("i", "x_i", "y_i = f(x_i)"))
    print("-" * 50)

    # Печатаем исходную таблицу узлов и значений
    for i in range(len(x_nodes)):
        print("{:<5} {:<15.10f} {:<20.10f}".format(i, x_nodes[i], y_nodes[i]))

    print("-" * 50)

    # Шаг 3. Построение многочлена Лагранжа
    # Получаем строковую запись многочлена и его значение в точке X*
    lagrange_poly_text = lagrange_polynomial_string(x_nodes, y_nodes)
    lagrange_at_x_star = lagrange_value(x_nodes, y_nodes, x_star)

    print("\nМногочлен Лагранжа:")
    print(f"L(x) = {lagrange_poly_text}")

    # Шаг 4. Построение таблицы разделённых разностей
    # Она нужна для построения многочлена Ньютона
    diff_table = divided_differences_table(x_nodes, y_nodes)
    print()
    print_divided_differences_table(x_nodes, diff_table)
```

```
    # Шаг 5. Построение многочлена Ньютона
    # Получаем строковую запись многочлена и его значение в точке X*
    newton_poly_text = newton_polynomial_string(x_nodes, diff_table)
    newton_at_x_star = newton_value(x_nodes, diff_table, x_star)

    print("\nМногочлен Ньютона:")
    print(f"P(x) = {newton_poly_text}")

    # Шаг 6. Вычисление точного значения и погрешности
    # Сначала считаем точное значение исходной функции в точке X*
    y_exact = f(x_star)

    # Затем считаем абсолютные погрешности для Лагранжа и Ньютона
    error_lagrange = fabs(y_exact - lagrange_at_x_star)
    error_newton = fabs(y_exact - newton_at_x_star)

    print("\nВычисления в точке X*:")
    print(f"X* = {x_star:.10f}")
    print(f"Точное значение y(X*) = f(X*) = {y_exact:.10f}")
    print(f"L(X*) = {lagrange_at_x_star:.10f}")
    print(f"P(X*) = {newton_at_x_star:.10f}")
    print(f"|y(X*) - L(X*)| = {error_lagrange:.10f}")
    print(f"|y(X*) - P(X*)| = {error_newton:.10f}")

    print("\nВывод:")
    print("Для одного и того же набора узлов многочлены Лагранжа и Ньютона")
    print("задают один и тот же интерполяционный многочлен, поэтому значения")
    print("L(X*) и P(X*) совпадают с точностью до округления.")
    print()


# 9. Главная программа
def main():
    # По условию точка, в которой нужно вычислить значение интерполяции
    # и погрешность
    x_star = 0.8

    # Вариант 24
    # а) первый набор узлов
    x_nodes_a = [0.1, 0.5, 0.9, 1.3]

    # б) второй набор узлов
    x_nodes_b = [0.1, 0.5, 1.1, 1.3]

    # Решаем пункт а)
    solve_case("а", x_nodes_a, x_star)

    # Решаем пункт б)
    solve_case("б", x_nodes_b, x_star)


# Стандартная точка входа в программу:
# если файл запускается напрямую, выполняется main()
if __name__ == "__main__":
    main()
```

Вывод программы:

```
================================================================================
 ↪  ========================================
```

```
СЛУЧАЙ а
================================================================================
 ↪  ========================================
Таблица значений функции:
--------------------------------------------------
i     x_i             y_i = f(x_i)
--------------------------------------------------
0     0.1000000000    100.0000000000
1     0.5000000000    4.0000000000
2     0.9000000000    1.2345679012
3     1.3000000000    0.5917159763
--------------------------------------------------

Многочлен Лагранжа:
L(x) = -260.4166666667 * (x - 0.5) * (x - 0.9) * (x - 1.3) +31.2500000000 * (x
 ↪  - 0.1) * (x - 0.9) * (x - 1.3) -9.6450617284 * (x - 0.1) * (x - 0.5) * (x -
 ↪  1.3) +1.5409270217 * (x - 0.1) * (x - 0.5) * (x - 0.9)

Таблица разделённых разностей:
--------------------------------------------------------------------------------
 ↪  ----------------------------------------
i     x_i           f[x_i]              f[x_i, x_{i+1}]        f[x_i, x_{i+1},
 ↪  x_{i+2}] f[x_i, ..., x_{i+3}]
--------------------------------------------------------------------------------
 ↪  ----------------------------------------
0     0.1000000000 100.0000000000     -240.0000000000        291.3580246914
 ↪        -237.2708013734
1     0.5000000000 4.0000000000       -6.9135802469          6.6330630433
2     0.9000000000 1.2345679012       -1.6071298123
3     1.3000000000 0.5917159763
--------------------------------------------------------------------------------
 ↪  ----------------------------------------

Многочлен Ньютона:
P(x) = 100.0000000000 -240.0000000000 * (x - 0.1) +291.3580246914 * (x - 0.1)
 ↪  * (x - 0.5) -237.2708013734 * (x - 0.1) * (x - 0.5) * (x - 0.9)

Вычисления в точке X*:
X* = 0.8000000000
Точное значение y(X*) = f(X*) = 1.5625000000
L(X*) = -1.8321279860
P(X*) = -1.8321279860
|y(X*) - L(X*)| = 3.3946279860
|y(X*) - P(X*)| = 3.3946279860

Вывод:
Для одного и того же набора узлов многочлены Лагранжа и Ньютона
задают один и тот же интерполяционный многочлен, поэтому значения
L(X*) и P(X*) совпадают с точностью до округления.

================================================================================
 ↪  ========================================
СЛУЧАЙ б
================================================================================
 ↪  ========================================
Таблица значений функции:
--------------------------------------------------
i     x_i             y_i = f(x_i)
--------------------------------------------------
0     0.1000000000    100.0000000000
```

```
1     0.5000000000    4.0000000000
2     1.1000000000    0.8264462810
3     1.3000000000    0.5917159763
-------------------------------------------------

Многочлен Лагранжа:
L(x) = -208.3333333333 * (x - 0.5) * (x - 1.1) * (x - 1.3) +20.8333333333 * (x
 ↪  - 0.1) * (x - 1.1) * (x - 1.3) -6.8870523416 * (x - 0.1) * (x - 0.5) * (x -
 ↪  1.3) +3.0818540434 * (x - 0.1) * (x - 0.5) * (x - 1.1)

Таблица разделённых разностей:
-------------------------------------------------------------------------------
 ↪  ----------------------------------------
i     x_i          f[x_i]             f[x_i, x_{i+1}]        f[x_i, x_{i+1},
 ↪  x_{i+2}] f[x_i, ..., x_{i+3}]
-------------------------------------------------------------------------------
 ↪  ----------------------------------------
0     0.1000000000 100.0000000000     -240.0000000000        234.7107438017
 ↪        -191.3051982982
1     0.5000000000 4.0000000000       -5.2892561983          5.1445058438
2     1.1000000000 0.8264462810       -1.1736515233
3     1.3000000000 0.5917159763
-------------------------------------------------------------------------------
 ↪  ----------------------------------------

Многочлен Ньютона:
P(x) = 100.0000000000 -240.0000000000 * (x - 0.1) +234.7107438017 * (x - 0.1)
 ↪  * (x - 0.5) -191.3051982982 * (x - 0.1) * (x - 0.5) * (x - 1.1)

Вычисления в точке X*:
X* = 0.8000000000
Точное значение y(X*) = f(X*) = 1.5625000000
L(X*) = -6.6585163089
P(X*) = -6.6585163089
|y(X*) - L(X*)| = 8.2210163089
|y(X*) - P(X*)| = 8.2210163089

Вывод:
Для одного и того же набора узлов многочлены Лагранжа и Ньютона
задают один и тот же интерполяционный многочлен, поэтому значения
L(X*) и P(X*) совпадают с точностью до округления.
```

3.1.5. Анализ результатов

Для каждого набора узлов формы Лагранжа и Ньютона совпали с точностью округления. В случае а) абсолютная ошибка в точке X^* составила 3.3946279860, а в случае б) --- 8.2210163089. Поэтому для данной точки первый набор узлов даёт более точную интерполяцию.

3.2. Задание 3.2

3.2.1. Формулировка задания

Построить естественный кубический сплайн по заданной таблице значений и вычислить значение сплайна в указанной точке.

3.2.2. Вариант № 24

x_i	0.1	0.5	0.9	1.3	1.7
f_i	100.00000	4.00000	1.23460	0.59172	0.34602

, $X^* = 0.8.$

3.2.3. Основные формулы

На отрезке $[x_{i-1}, x_i]$ сплайн имеет вид

$$S_i(x) = a_i + b_i(x - x_{i-1}) + c_i(x - x_{i-1})^2 + d_i(x - x_{i-1})^3.$$

Коэффициенты находятся из условий интерполяции и гладкости, а для естественного сплайна дополнительно задаются условия

$$S''(x_0) = 0, \qquad S''(x_n) = 0.$$

3.2.4. Текст программы

```
from typing import List, Tuple


def solve_linear_system_gauss(a: List[List[float]], b: List[float]) ->
↪  List[float]:
    """
    Решение СЛАУ методом Гаусса с частичным выбором главного элемента.
    Используется только для решения системы (3.13).
    """
    n = len(a)

    # Формируем расширенную матрицу
    aug = [row[:] + [b[i]] for i, row in enumerate(a)]

    # Прямой ход
    for k in range(n):
        # Выбор главного элемента в столбце k
        pivot_row = k
        max_value = abs(aug[k][k])
        for i in range(k + 1, n):
            if abs(aug[i][k]) > max_value:
                max_value = abs(aug[i][k])
                pivot_row = i

        if max_value == 0:
            raise ValueError("Система не имеет единственного решения: нулевой
            ↪  ведущий элемент.")

        if pivot_row != k:
            aug[k], aug[pivot_row] = aug[pivot_row], aug[k]

        # Обнуление элементов ниже главного
        for i in range(k + 1, n):
            factor = aug[i][k] / aug[k][k]
            for j in range(k, n + 1):
                aug[i][j] -= factor * aug[k][j]

    # Обратный ход
    x = [0.0] * n
```

```
    for i in range(n - 1, -1, -1):
        s = 0.0
        for j in range(i + 1, n):
            s += aug[i][j] * x[j]
        if aug[i][i] == 0:
            raise ValueError("Система не имеет единственного решения: нулевой
            ↪  диагональный элемент.")
        x[i] = (aug[i][n] - s) / aug[i][i]

    return x


def build_natural_cubic_spline(x: List[float], f: List[float]) ->
↪  Tuple[List[float], List[float], List[float], List[float],
↪  List[List[float]], List[float], List[float]]:
    """
    Построение кубического сплайна по формулам (3.13) и (3.14)
    для случая нулевой кривизны на концах отрезка.

    Возвращает коэффициенты a_i, b_i, c_i, d_i для интервалов:
    S_i(x) = a_i + b_i (x - x_{i-1}) + c_i (x - x_{i-1})^2 + d_i (x -
    ↪  x_{i-1})^3,
    i = 1, ..., n.
    """
    if len(x) != len(f):
        raise ValueError("Списки x и f должны иметь одинаковую длину.")

    n = len(x) - 1  # число интервалов
    if n < 1:
        raise ValueError("Для построения сплайна нужно минимум две точки.")

    # Проверка строгого возрастания узлов и вычисление h_i
    h = []
    for i in range(1, len(x)):
        hi = x[i] - x[i - 1]
        if hi <= 0:
            raise ValueError("Узлы x должны быть строго возрастающими.")
        h.append(hi)

    # Если интервал один, строим линейный сплайн как частный случай
    if n == 1:
        a = [f[0]]
        b = [(f[1] - f[0]) / h[0]]
        c = [0.0]
        d = [0.0]
        return a, b, c, d, A, rhs, c_full, [], [], [0.0, 0.0]

    # По условию естественного сплайна:
    # c1 = 0, c_{n+1} = 0
    # Неизвестные: c2, c3, ..., c_n
    size = n - 1
    A = [[0.0 for _ in range(size)] for _ in range(size)]
    rhs = [0.0 for _ in range(size)]

    # Система (3.13)
    for row in range(size):
        i = row + 1  # соответствует внутреннему узлу x_i, i = 1..n-1 в
        ↪  нумерации Python

        if row - 1 >= 0:
```

```
            A[row][row - 1] = h[i - 1]

        A[row][row] = 2.0 * (h[i - 1] + h[i])

        if row + 1 < size:
            A[row][row + 1] = h[i]

        rhs[row] = 3.0 * ((f[i + 1] - f[i]) / h[i] - (f[i] - f[i - 1]) / h[i -
        ↪  1])

    internal_c = solve_linear_system_gauss(A, rhs)

    # Полный список c: [c1, c2, ..., c_n, c_{n+1}]
    c_full = [0.0] + internal_c + [0.0]

    # Восстановление коэффициентов по формулам (3.14)
    a = [0.0] * n
    b = [0.0] * n
    c = [0.0] * n
    d = [0.0] * n

    for i in range(1, n + 1):
        hi = h[i - 1]
        a[i - 1] = f[i - 1]
        c[i - 1] = c_full[i - 1]

        if i < n:
            b[i - 1] = (f[i] - f[i - 1]) / hi - hi * (c_full[i] + 2.0 *
            ↪  c_full[i - 1]) / 3.0
        else:
            b[i - 1] = (f[i] - f[i - 1]) / hi - 2.0 * hi * c_full[i - 1] / 3.0

        d[i - 1] = (c_full[i] - c_full[i - 1]) / (3.0 * hi)

    return a, b, c, d, A, rhs, c_full


def evaluate_spline(x_nodes: List[float], a: List[float], b: List[float], c:
↪  List[float], d: List[float], x_star: float) -> Tuple[float, int]:
    """
    Вычисление значения сплайна в точке x_star.
    Возвращает значение и номер интервала (нумерация с 1).
    """
    n = len(x_nodes) - 1

    if not (x_nodes[0] <= x_star <= x_nodes[-1]):
        raise ValueError("Точка x* должна принадлежать отрезку [x0, xn].")

    # Поиск интервала [x_{i-1}, x_i]
    interval = None
    for i in range(1, n + 1):
        if x_nodes[i - 1] <= x_star <= x_nodes[i]:
            interval = i
            break

    if interval is None:
        raise ValueError("Не удалось определить интервал для точки x*.")

    dx = x_star - x_nodes[interval - 1]
    value = a[interval - 1] + b[interval - 1] * dx + c[interval - 1] * dx ** 2
    ↪  + d[interval - 1] * dx ** 3
```

```
    return value, interval


def print_table(x: List[float], a: List[float], b: List[float], c:
↪  List[float], d: List[float]) -> None:
    print("\nКоэффициенты кубического сплайна:")
    print("-" * 106)
    print(f"{'i':>2} | {'[x_(i-1), x_i]':>20} | {'a_i':>14} | {'b_i':>14} |
    ↪  {'c_i':>14} | {'d_i':>14}")
    print("-" * 106)
    for i in range(1, len(a) + 1):
        interval_str = f"[{x[i - 1]:.6f}, {x[i]:.6f}]"
        print(
            f"{i:>2} | {interval_str:>20} | "
            f"{a[i - 1]:>14.8f} | {b[i - 1]:>14.8f} | {c[i - 1]:>14.8f} | {d[i
            ↪  - 1]:>14.8f}"
        )
    print("-" * 106)


def main() -> None:
    print("ЛР 3.2, вариант 24")
    print("Построение кубического сплайна с нулевой кривизной на концах
    ↪  отрезка")

    # Данные варианта 24
    x = [0.1, 0.5, 0.9, 1.3, 1.7]
    f = [100.00, 4.0, 1.2346, 0.59172, 0.34602]
    x_star = 0.8

    print("\nИсходные данные:")
    print("x_i =", x)
    print("f_i =", f)
    print(f"x* = {x_star}")

    # Шаги h_i
    h = [x[i] - x[i - 1] for i in range(1, len(x))]
    print("\nШаги h_i:")
    for i, hi in enumerate(h, start=1):
        print(f"h{i} = x{i} - x{i - 1} = {x[i]} - {x[i - 1]} = {hi:.8f}")

    # Построение сплайна
    a, b, c, d, A, rhs, c_full = build_natural_cubic_spline(x, f)


    print("\nСистема для коэффициентов c_i по формуле (3.13):")
    if A:
        for i in range(len(A)):
            row_str = " + ".join(f"({A[i][j]:.8f})*c{j + 2}" for j in
            ↪  range(len(A[i])) if abs(A[i][j]) > 1e-15)
            print(f"{row_str} = {rhs[i]:.8f}")
    else:
        print("Внутренних коэффициентов нет, так как интервал только один.")

    print("\nКоэффициенты c с учетом условий нулевой кривизны:")
    for idx, value in enumerate(c_full, start=1):
        print(f"c{idx} = {value:.8f}")

    print_table(x, a, b, c, d)
```

```
    # Вычисление значения в точке x*
    y_star, interval = evaluate_spline(x, a, b, c, d, x_star)
    left = x[interval - 1]
    right = x[interval]

    print(f"\nТочка x* = {x_star} принадлежит интервалу [{left}, {right}] (i =
    ↪  {interval}).")
    print("На этом интервале сплайн имеет вид:")
    print(
        f"S_{interval}(x) = {a[interval - 1]:.8f}"
        f" + ({b[interval - 1]:.8f}) * (x - {left:.8f})"
        f" + ({c[interval - 1]:.8f}) * (x - {left:.8f})^2"
        f" + ({d[interval - 1]:.8f}) * (x - {left:.8f})^3"
    )

    print(f"\nЗначение сплайна в точке x* = {x_star}:")
    print(f"S({x_star}) ≈ {y_star:.10f}")


if __name__ == "__main__":
    main()
```

Вывод программы:

```
ЛР 3.2, вариант 24
Построение кубического сплайна с нулевой кривизной на концах отрезка

Исходные данные:
x_i = [0.1, 0.5, 0.9, 1.3, 1.7]
f_i = [100.0, 4.0, 1.2346, 0.59172, 0.34602]
x* = 0.8

Шаги h_i:
h1 = x1 - x0 = 0.5 - 0.1 = 0.40000000
h2 = x2 - x1 = 0.9 - 0.5 = 0.40000000
h3 = x3 - x2 = 1.3 - 0.9 = 0.40000000
h4 = x4 - x3 = 1.7 - 1.3 = 0.40000000

Система для коэффициентов c_i по формуле (3.13):
(1.60000000)*c2 + (0.40000000)*c3 = 699.25950000
(0.40000000)*c2 + (1.60000000)*c3 + (0.40000000)*c4 = 15.91890000
(0.40000000)*c3 + (1.60000000)*c4 = 2.97885000

Коэффициенты c с учетом условий нулевой кривизны:
c1 = 0.00000000
c2 = 465.54445312
c3 = -114.02906250
c4 = 30.36904688
c5 = 0.00000000

Коэффициенты кубического сплайна:
-------------------------------------------------------------------------------
 ↪  ----------------------------
 i |       [x_(i-1), x_i] |           a_i |           b_i |           c_i |
  ↪          d_i
-------------------------------------------------------------------------------
 ↪  ----------------------------
 1 | [0.100000, 0.500000] |   100.00000000 |  -302.07259375 |     0.00000000 |
  ↪   387.95371094
```

```
 2 | [0.500000, 0.900000] |     4.00000000 |  -115.85481250 |   465.54445312 |
 ↪  -482.97792969
 3 | [0.900000, 1.300000] |     1.23460000 |    24.75134375 |  -114.02906250 |
 ↪   120.33175781
 4 | [1.300000, 1.700000] |     0.59172000 |    -8.71266250 |    30.36904688 |
 ↪   -25.30753906
-------------------------------------------------------------------------------
↪  ---------------------------

Точка x* = 0.8 принадлежит интервалу [0.5, 0.9] (i = 2).
На этом интервале сплайн имеет вид:
S_2(x) = 4.00000000 + (-115.85481250) * (x - 0.50000000) + (465.54445312) * (x
↪  - 0.50000000)^2 + (-482.97792969) * (x - 0.50000000)^3

Значение сплайна в точке x* = 0.8:
S(0.8) ≈ -1.8978470703
```

3.2.5. Анализ результатов

Точка $X^* = 0.8$ находится на отрезке $[0.5, 0.9]$. Для найденных коэффициентов программа получила

$$S(0.8) = -1.8978470703.$$

3.3. Задание 3.3

3.3.1. Формулировка задания

Методом наименьших квадратов построить приближающие многочлены первой и второй степени для таблично заданной функции. Сравнить суммы квадратов ошибок и построить график.

3.3.2. Вариант № 24

x_i	0.1	0.5	0.9	1.3	1.7	2.1
y_i	100.00000	4.00000	1.23460	0.59172	0.34602	0.22676

.

3.3.3. Основные формулы

Для $F_m(x) = a_0 + a_1 x + \ldots + a_m x^m$ коэффициенты находятся из условия

$$\Phi(a_0, \ldots, a_m) = \sum_{i=0}^{n} (F_m(x_i) - y_i)^2 \longrightarrow \min\,.$$

Нормальная система имеет вид

$$\sum_{j=0}^{m} a_j \sum_{i=0}^{n} x_i^{j+k} = \sum_{i=0}^{n} y_i x_i^k, \qquad k = 0, \ldots, m.$$

3.3.4. Текст программы

```
import numpy as np
import matplotlib.pyplot as plt

# ===============================================================================
```

```
# Лабораторная работа 3.3 (по выданному файлу задания)
# Вариант 24
#
# По содержанию задания требуется:
# 1) методом наименьших квадратов найти приближающий многочлен 1-й степени;
# 2) методом наименьших квадратов найти приближающий многочлен 2-й степени;
# 3) для каждого многочлена вычислить сумму квадратов ошибок;
# 4) построить графики исходной табличной функции и найденных многочленов.
# ============================================================================


def print_matrix(A, title):
    """
    Красивый вывод матрицы в консоль.

    Зачем нужна эта функция:
    - в отчёте и на защите удобно показать саму нормальную систему;
    - видно, из каких чисел она составилась после подсчёта сумм;
    - можно проверить, что мы действительно работаем по алгоритму МНК,
      а не просто подставили ответ.

    Параметры:
    A     -- двумерный массив NumPy, то есть матрица системы
    title -- заголовок перед матрицей
    """
    print(title)
    for row in A:
        print("  ", " ".join(f"{value:14.8f}" for value in row))
    print()



def print_vector(b, title):
    """
    Красивый вывод вектора правой части системы.

    Правая часть нормальной системы в МНК состоит из сумм вида:
    Σy, Σxy, Σx^2 y и т.д. в зависимости от степени многочлена.
    Эта функция нужна только для удобного и наглядного вывода.
    """
    print(title)
    for value in b:
        print(f"  {value:14.8f}")
    print()



def least_squares_linear(x, y):
    """
    Находит приближающий многочлен 1-й степени методом наименьших квадратов.

    Ищем многочлен вида:
        F1(x) = a0 + a1*x

    По методичке коэффициенты a0 и a1 находятся из нормальной системы:

        [N+1   Σx   ] [a0] = [Σy]
        [Σx   Σx^2 ] [a1]   [Σxy]

    Здесь N = len(x) - 1, потому что в методичке индексация идёт от 0 до N,
```

```
то есть всего точек N + 1.

Возвращаем:
coeffs -- коэффициенты [a0, a1]
f      -- значения F1(x_i) в исходных узлах
phi    -- сумма квадратов ошибок Φ = Σ(F1(x_i) - y_i)^2
A      -- матрица нормальной системы
b      -- правая часть системы
sums   -- словарь с промежуточными суммами для удобства вывода
"""

# Количество узлов в обозначениях методички: i = 0..N.
# Если точек 6, то N = 5.
n = len(x) - 1

# ---------------------------------------------------------------------------
# Считаем суммы, которые напрямую входят в нормальную систему.
# Это ровно тот шаг, который в методичке делается перед составлением СЛАУ.
# ---------------------------------------------------------------------------
sx = np.sum(x)           # Σx
sx2 = np.sum(x ** 2)     # Σx^2
sy = np.sum(y)           # Σy
sxy = np.sum(x * y)      # Σxy

# ---------------------------------------------------------------------------
# Собираем матрицу A и вектор b нормальной системы A * a = b.
# После этого задача МНК сводится к решению обычной системы линейных
# алгебраических уравнений относительно неизвестных коэффициентов a0, a1.
# ---------------------------------------------------------------------------
A = np.array([
    [n + 1, sx],
    [sx, sx2]
], dtype=float)

b = np.array([sy, sxy], dtype=float)

# Решаем систему и получаем коэффициенты прямой,
# которая лучше всего приближает данные в смысле МНК.
coeffs = np.linalg.solve(A, b)
a0, a1 = coeffs

# Вычисляем значения найденного многочлена в исходных узлах x_i.
# Это понадобится для подсчёта ошибок и вывода таблицы.
f = a0 + a1 * x

# Функционал МНК: сумма квадратов отклонений от табличных значений.
# Чем меньше Φ, тем лучше приближение по критерию МНК.
phi = np.sum((f - y) ** 2)

sums = {
    'N': n,
    'sum_x': sx,
    'sum_x2': sx2,
    'sum_y': sy,
    'sum_xy': sxy,
}

return coeffs, f, phi, A, b, sums
```

```
def least_squares_quadratic(x, y):
    """
    Находит приближающий многочлен 2-й степени методом наименьших квадратов.

    Ищем многочлен вида:
        F2(x) = a0 + a1*x + a2*x^2

    По методичке коэффициенты a0, a1, a2 находятся из нормальной системы:

        [N+1   Σx    Σx^2 ] [a0] = [Σy    ]
        [Σx   Σx^2   Σx^3 ] [a1]   [Σxy   ]
        [Σx^2 Σx^3   Σx^4 ] [a2]   [Σx^2y ]

    Это уже система 3x3, потому что неизвестных коэффициентов три.
    """

    # Число N в обозначениях методички.
    n = len(x) - 1

    # ---------------------------------------------------------------------
    # Считаем все суммы, которые понадобятся для составления нормальной системы
    # второго порядка.
    # ---------------------------------------------------------------------
    sx = np.sum(x)                  # Σx
    sx2 = np.sum(x ** 2)            # Σx^2
    sx3 = np.sum(x ** 3)            # Σx^3
    sx4 = np.sum(x ** 4)            # Σx^4
    sy = np.sum(y)                  # Σy
    sxy = np.sum(x * y)             # Σxy
    sx2y = np.sum((x ** 2) * y)     # Σx^2 y

    # Матрица нормальной системы для квадратичного многочлена.
    A = np.array([
        [n + 1, sx, sx2],
        [sx, sx2, sx3],
        [sx2, sx3, sx4]
    ], dtype=float)

    # Правая часть системы.
    b = np.array([sy, sxy, sx2y], dtype=float)

    # Решаем систему и получаем коэффициенты a0, a1, a2.
    coeffs = np.linalg.solve(A, b)
    a0, a1, a2 = coeffs

    # Находим значения многочлена F2 в исходных точках таблицы.
    f = a0 + a1 * x + a2 * x ** 2

    # Сумма квадратов ошибок для многочлена 2-й степени.
    phi = np.sum((f - y) ** 2)

    sums = {
        'N': n,
        'sum_x': sx,
        'sum_x2': sx2,
        'sum_x3': sx3,
        'sum_x4': sx4,
        'sum_y': sy,
        'sum_xy': sxy,
```

```
        'sum_x2y': sx2y,
    }

    return coeffs, f, phi, A, b, sums



def main():
    """
    Основная функция программы.

    Здесь последовательно реализуются все шаги решения лабораторной работы:
    1. Задаются исходные табличные данные варианта.
    2. Строится МНК-многочлен 1-й степени.
    3. Строится МНК-многочлен 2-й степени.
    4. Для каждого считается сумма квадратов ошибок.
    5. Сравнивается качество приближения.
    6. Строится общий график.
    """

    # ------------------------------------------------------------------------
    # Исходные данные варианта 24 из файла задания.
    # x -- узлы таблицы
    # y -- значения функции в этих узлах
    # ------------------------------------------------------------------------
    x = np.array([0.1, 0.5, 0.9, 1.3, 1.7, 2.1], dtype=float)
    y = np.array([100.0, 4.0, 1.2346, 0.59172, 0.34602, 0.22676], dtype=float)

    print("ЛР 3.3, вариант 24")
    print("Метод: МНК через нормальные системы из методички")
    print()

    # Выводим исходную таблицу, чтобы было видно, с какими данными работаем.
    print("Исходные данные:")
    for i in range(len(x)):
        print(f"i = {i:>1d}; x[{i}] = {x[i]:8.5f}; y[{i}] = {y[i]:10.5f}")
    print()

    # ======================== МНОГОЧЛЕН 1-Й СТЕПЕНИ ========================
    # На этом шаге строится линейное приближение: F1(x) = a0 + a1*x.
    coeffs1, f1, phi1, A1, b1, sums1 = least_squares_linear(x, y)
    a0_1, a1_1 = coeffs1

    print("=" * 90)
    print("ПРИБЛИЖАЮЩИЙ МНОГОЧЛЕН 1-Й СТЕПЕНИ")
    print("F1(x) = a0 + a1*x")
    print()

    # Показываем все суммы, из которых составлена нормальная система.
    print("Суммы для нормальной системы:")
    print(f"N = {sums1['N']}")
    print(f"Σx    = {sums1['sum_x']:.8f}")
    print(f"Σx^2  = {sums1['sum_x2']:.8f}")
    print(f"Σy    = {sums1['sum_y']:.8f}")
    print(f"Σxy   = {sums1['sum_xy']:.8f}")
    print()

    # Выводим саму систему в матричной форме.
    print_matrix(A1, "Матрица нормальной системы для F1:")
    print_vector(b1, "Правая часть системы для F1:")
```

```
# Печатаем найденные коэффициенты и сам итоговый многочлен.
print("Решение системы:")
print(f"a0 = {a0_1:.8f}")
print(f"a1 = {a1_1:.8f}")
print()
print(f"Итоговый многочлен: F1(x) = {a0_1:.8f} + ({a1_1:.8f})*x")
print()

# Для каждой табличной точки считаем:
# 1) значение приближающего многочлена F1(x_i)
# 2) квадрат ошибки (F1(x_i) - y_i)^2
# Это позволяет затем получить сумму квадратов ошибок Ф1.
print("Таблица значений F1(x_i) и квадратов ошибок:")
print(f"{'i':>2} {'x_i':>10} {'y_i':>12} {'F1(x_i)':>14} {'(F1-y)^2':>16}")
for i in range(len(x)):
    err2 = (f1[i] - y[i]) ** 2
    print(f"{i:>2d} {x[i]:>10.5f} {y[i]:>12.5f} {f1[i]:>14.8f}
    ↪  {err2:>16.8f}")
print()
print(f"Сумма квадратов ошибок Ф1 = {phi1:.8f}")
print()

# ======================== МНОГОЧЛЕН 2-Й СТЕПЕНИ ========================
# Теперь строим квадратичное приближение:
# F2(x) = a0 + a1*x + a2*x^2.
coeffs2, f2, phi2, A2, b2, sums2 = least_squares_quadratic(x, y)
a0_2, a1_2, a2_2 = coeffs2

print("=" * 90)
print("ПРИБЛИЖАЮЩИЙ МНОГОЧЛЕН 2-Й СТЕПЕНИ")
print("F2(x) = a0 + a1*x + a2*x^2")
print()

# Снова выводим все промежуточные суммы, но уже для системы 3x3.
print("Суммы для нормальной системы:")
print(f"Σx      = {sums2['sum_x']:.8f}")
print(f"Σx^2    = {sums2['sum_x2']:.8f}")
print(f"Σx^3    = {sums2['sum_x3']:.8f}")
print(f"Σx^4    = {sums2['sum_x4']:.8f}")
print(f"Σy      = {sums2['sum_y']:.8f}")
print(f"Σxy     = {sums2['sum_xy']:.8f}")
print(f"Σx^2*y  = {sums2['sum_x2y']:.8f}")
print()

print_matrix(A2, "Матрица нормальной системы для F2:")
print_vector(b2, "Правая часть системы для F2:")

# Печатаем коэффициенты квадратичного многочлена.
print("Решение системы:")
print(f"a0 = {a0_2:.8f}")
print(f"a1 = {a1_2:.8f}")
print(f"a2 = {a2_2:.8f}")
print()
print(f"Итоговый многочлен: F2(x) = {a0_2:.8f} + ({a1_2:.8f})*x +
↪  ({a2_2:.8f})*x^2")
print()

# Аналогично линейному случаю строим таблицу ошибок для F2.
print("Таблица значений F2(x_i) и квадратов ошибок:")
```

```
    print(f"{'i':>2} {'x_i':>10} {'y_i':>12} {'F2(x_i)':>14} {'(F2-y)^2':>16}")
    for i in range(len(x)):
        err2 = (f2[i] - y[i]) ** 2
        print(f"{i:>2d} {x[i]:>10.5f} {y[i]:>12.5f} {f2[i]:>14.8f}
         ↪  {err2:>16.8f}")
    print()
    print(f"Сумма квадратов ошибок Ф2 = {phi2:.8f}")
    print()

    # ---------------------------------------------------------------------
    # Сравниваем два найденных приближения.
    # По критерию метода наименьших квадратов лучше тот многочлен,
    # у которого сумма квадратов ошибок меньше.
    # ---------------------------------------------------------------------
    print("=" * 90)
    if phi2 < phi1:
        print("Лучшее приближение по критерию МНК дает многочлен 2-й степени,")
        print("так как для него сумма квадратов ошибок меньше.")
    else:
        print("Лучшее приближение по критерию МНК дает многочлен 1-й степени,")
        print("так как для него сумма квадратов ошибок меньше.")
    print("=" * 90)

    # ============================== ГРАФИК ================================
    # Для красивого графика строим плотную сетку из 400 точек.
    # На ней считаем значения обоих многочленов.
    x_dense = np.linspace(np.min(x), np.max(x), 400)
    y1_dense = a0_1 + a1_1 * x_dense
    y2_dense = a0_2 + a1_2 * x_dense + a2_2 * x_dense ** 2

    # Создаём рисунок.
    plt.figure(figsize=(10, 6))

    # Исходные табличные данные показываем точками.
    plt.scatter(x, y, label='Табличные данные')

    # Линейное приближение рисуем линией.
    plt.plot(x_dense, y1_dense, label='МНК, 1-я степень')

    # Квадратичное приближение рисуем отдельной линией.
    plt.plot(x_dense, y2_dense, label='МНК, 2-я степень')

    # Оформление графика.
    plt.xlabel('x')
    plt.ylabel('y')
    plt.title('ЛР 3.3, вариант 24: МНК')
    plt.grid(True)
    plt.legend()
    plt.tight_layout()

    # Сохраняем график в файл, чтобы его можно было вставить в отчёт.
    plt.savefig('3_3.png', dpi=150)

    # Показываем график на экране.
    plt.show()


# Эта проверка означает:
# код внутри блока ниже выполнится только тогда,
# когда файл запускают как основную программу.
```

```
# Это стандартная практика в Python.
if __name__ == '__main__':
    main()
```

Вывод программы:

```
ЛР 3.3, вариант 24
Метод: МНК через нормальные системы из методички

Исходные данные:
i = 0; x[0] =  0.10000; y[0] =  100.00000
i = 1; x[1] =  0.50000; y[1] =    4.00000
i = 2; x[2] =  0.90000; y[2] =    1.23460
i = 3; x[3] =  1.30000; y[3] =    0.59172
i = 4; x[4] =  1.70000; y[4] =    0.34602
i = 5; x[5] =  2.10000; y[5] =    0.22676

================================================================================
 ↪  =============
ПРИБЛИЖАЮЩИЙ МНОГОЧЛЕН 1-Й СТЕПЕНИ
F1(x) = a0 + a1*x

Суммы для нормальной системы:
N = 5
Σx    = 6.60000000
Σx^2  = 10.06000000
Σy    = 106.39910000
Σxy   = 14.94480600

Матрица нормальной системы для F1:
       6.00000000     6.60000000
       6.60000000    10.06000000

Правая часть системы для F1:
    106.39910000
     14.94480600

Решение системы:
a0 = 57.84162062
a1 = -36.46221571

Итоговый многочлен: F1(x) = 57.84162062 + (-36.46221571)*x

Таблица значений F1(x_i) и квадратов ошибок:
 i        x_i          y_i        F1(x_i)         (F1-y)^2
 0    0.10000    100.00000    54.19539905    2098.06146841
 1    0.50000      4.00000    39.61051276    1268.10861917
 2    0.90000      1.23460    25.02562648     566.01294079
 3    1.30000      0.59172    10.44074019      97.00319871
 4    1.70000      0.34602    -4.14414610      20.16159156
 5    2.10000      0.22676   -18.72903238     359.32206479

Сумма квадратов ошибок Ф1 = 4408.66988343

================================================================================
 ↪  =============
ПРИБЛИЖАЮЩИЙ МНОГОЧЛЕН 2-Й СТЕПЕНИ
F2(x) = a0 + a1*x + a2*x^2
```

```
Суммы для нормальной системы:
Σx      = 6.60000000
Σx^2    = 10.06000000
Σx^3    = 17.22600000
Σx^4    = 31.37500000
Σy      = 106.39910000
Σxy     = 14.94480600
Σx^2*y  = 6.00004220

Матрица нормальной системы для F2:
       6.00000000     6.60000000    10.06000000
       6.60000000    10.06000000    17.22600000
      10.06000000    17.22600000    31.37500000

Правая часть системы для F2:
    106.39910000
     14.94480600
      6.00004220

Решение системы:
a0 = 98.44972981
a1 = -156.64765098
a2 = 54.62974330

Итоговый многочлен: F2(x) = 98.44972981 + (-156.64765098)*x + (54.62974330)*x^2

Таблица значений F2(x_i) и квадратов ошибок:
 i        x_i          y_i        F2(x_i)         (F2-y)^2
 0    0.10000    100.00000    83.33126214     277.84682175
 1    0.50000      4.00000    33.78334014     887.04735007
 2    0.90000      1.23460     1.71693600       0.23264802
 3    1.30000      0.59172   -12.86795029     181.16272420
 4    1.70000      0.34602    -9.97131871     106.44747815
 5    2.10000      0.22676    10.40683071     103.63383975

Сумма квадратов ошибок Ф2 = 1556.37086193

================================================================================
 ↪  =============
Лучшее приближение по критерию МНК дает многочлен 2-й степени,
так как для него сумма квадратов ошибок меньше.
================================================================================
 ↪  =============
```

3.3.5. График аппроксимации

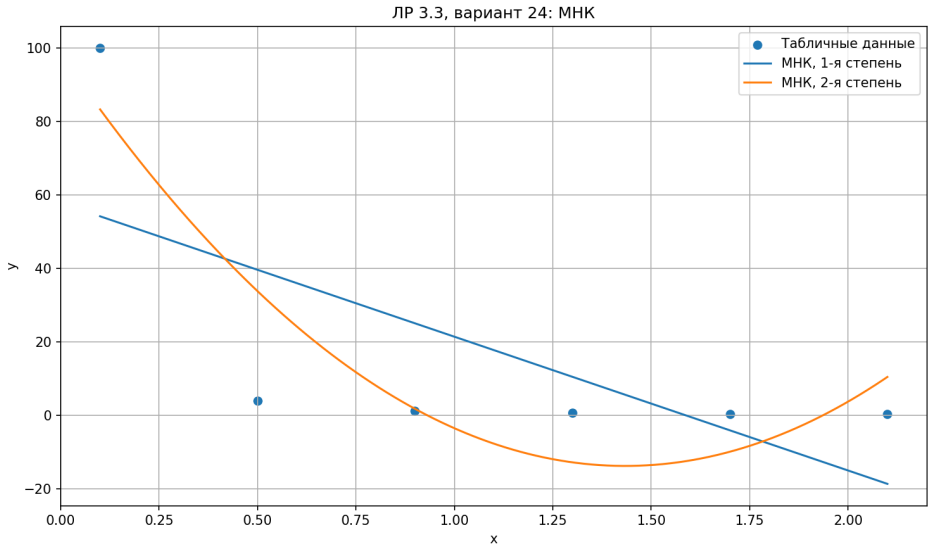


Рис. 3: Аппроксимация табличных данных методом наименьших квадратов

3.3.6. Анализ результатов

Получены многочлены

$$F_1(x) = 57.84162062 - 36.46221571x, \qquad \Phi_1 = 4408.66988343,$$

$$F_2(x) = 98.44972981 - 156.64765098x + 54.62974330x^2, \qquad \Phi_2 = 1556.37086193.$$

Так как $\Phi_2 < \Phi_1$, лучшим из исследованных приближений является многочлен второй степени.

3.4. Задание 3.4

3.4.1. Формулировка задания

По табличным значениям функции вычислить первую и вторую производные в указанной точке с использованием формул численного дифференцирования.

3.4.2. Вариант № 24

x_i	1.0	1.2	1.4	1.6	1.8
y_i	2.0000	2.1344	2.4702	2.9506	3.5486

, $\quad X^* = 1.4.$

3.4.3. Основные формулы

Для равномерной сетки с шагом h центральные формулы имеют вид

$$y'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2h}, \qquad y''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2}.$$

3.4.4. Текст программы

```
# Лабораторная работа 3.4
# Численное дифференцирование таблично заданной функции
# Вариант 24
#
# Цель:
# Вычислить первую и вторую производную функции, заданной таблицей,
# в точке X* = 1.4.
#
# Решение выполняется строго по формулам:
# 1) формула (3.18) — первая производная на отрезке;
# 2) формула (3.20) — первая производная со вторым порядком точности;
# 3) формула (3.21) — вторая производная.


EPS = 1e-12


def check_input(xs, ys, x_star):
    """
    Проверка исходных данных.

    Эти проверки нужны, чтобы формулы из методички можно было применять
    ↪ корректно:
    - количество x и y должно совпадать;
    - должно быть минимум 3 точки, потому что для формул (3.20) и (3.21)
      нужен интерполяционный многочлен второй степени, то есть три соседних
      ↪ узла;
    - узлы x должны идти строго по возрастанию, иначе возможны нулевые
      или некорректные знаменатели;
    - точка X* должна совпадать с одним из внутренних узлов таблицы,
      как в примере из методички;
    - у точки X* должен быть сосед слева и сосед справа.
    """

    if len(xs) != len(ys):
        raise ValueError("Ошибка: количество значений x и y должно совпадать.")

    if len(xs) < 3:
        raise ValueError("Ошибка: для вычислений нужно минимум 3 табличные
        ↪ точки.")

    for i in range(len(xs) - 1):
        if xs[i] >= xs[i + 1]:
            raise ValueError("Ошибка: значения x должны быть строго
            ↪ возрастающими.")

    target_index = None

    for i in range(len(xs)):
        if abs(xs[i] - x_star) < EPS:
            target_index = i
            break

    if target_index is None:
        raise ValueError("Ошибка: точка X* должна совпадать с одним из узлов
        ↪ таблицы.")

    if target_index == 0 or target_index == len(xs) - 1:
        raise ValueError(
```

```
            "Ошибка: точка X* не должна быть крайней, "
            "так как нужны соседний узел слева и соседний узел справа."
        )

    return target_index


def check_uniform_grid(xs):
    """
    Проверка равномерности сетки.

    В методичке указано, что формула (3.20) при равностоящих точках
    обеспечивает второй порядок точности.
    Для варианта 24 сетка равномерная: шаг h = 0.2.
    """

    h = xs[1] - xs[0]

    for i in range(1, len(xs) - 1):
        current_h = xs[i + 1] - xs[i]
        if abs(current_h - h) > EPS:
            return False, h

    return True, h


def calculate_derivatives(xs, ys, x_star):
    """
    Вычисление первой и второй производной в точке X*.

    Пусть X* = x_{i+1}.
    Тогда для формул берем три соседние точки:

    x_i     — точка слева от X*
    x_{i+1} — сама точка X*
    x_{i+2} — точка справа от X*

    Для моего варианта:
    x_i     = 1.2
    x_{i+1} = 1.4
    x_{i+2} = 1.6
    """

    target_index = check_input(xs, ys, x_star)

    # Так как X* = x_{i+1}, то индекс i равен target_index - 1.
    i = target_index - 1

    # Берем три соседние точки для формул (3.20) и (3.21).
    x_i = xs[i]
    x_i1 = xs[i + 1]
    x_i2 = xs[i + 2]

    y_i = ys[i]
    y_i1 = ys[i + 1]
    y_i2 = ys[i + 2]

    # Формула (3.18)
    # Левая производная:
    # y'(X*) ≈ (y_{i+1} - y_i) / (x_{i+1} - x_i)
```

```
    # Для моего варианта это производная на отрезке [1.2; 1.4].
    left_derivative = (y_i1 - y_i) / (x_i1 - x_i)

    # Правая производная:
    # y'(X*) ≈ (y_{i+2} - y_{i+1}) / (x_{i+2} - x_{i+1})
    #
    # Для моего варианта это производная на отрезке [1.4; 1.6].
    right_derivative = (y_i2 - y_i1) / (x_i2 - x_i1)

    # Формула (3.20)
    # Первая производная со вторым порядком точности.
    # В методичке формула имеет вид:
    # y'(x) ≈ (y_{i+1} - y_i) / (x_{i+1} - x_i)
    #       + [ ((y_{i+2} - y_{i+1}) / (x_{i+2} - x_{i+1})
    #          - (y_{i+1} - y_i) / (x_{i+1} - x_i))
    #          / (x_{i+2} - x_i) ]
    #       * (2x - x_i - x_{i+1})
    #
    # Здесь x = X*.
    first_derivative_second_order = left_derivative + (
        (right_derivative - left_derivative) / (x_i2 - x_i)
    ) * (2 * x_star - x_i - x_i1)

    # Формула (3.21)
    # Вторая производная.
    # y''(x) ≈ 2 * [ ((y_{i+2} - y_{i+1}) / (x_{i+2} - x_{i+1})
    #              - (y_{i+1} - y_i) / (x_{i+1} - x_i))
    #              / (x_{i+2} - x_i) ]
    second_derivative = 2 * (
        (right_derivative - left_derivative) / (x_i2 - x_i)
    )

    return {
        "i": i,
        "x_i": x_i,
        "x_i1": x_i1,
        "x_i2": x_i2,
        "y_i": y_i,
        "y_i1": y_i1,
        "y_i2": y_i2,
        "left_derivative": left_derivative,
        "right_derivative": right_derivative,
        "first_derivative_second_order": first_derivative_second_order,
        "second_derivative": second_derivative,
    }


def main():
    x_star = 1.4

    xs = [1.0, 1.2, 1.4, 1.6, 1.8]
    ys = [2.0, 2.1344, 2.4702, 2.9506, 3.5486]

    print("Лабораторная работа 3.4")
    print("Тема: численное дифференцирование")
    print("Вариант: 24")
    print()

    print("Исходные данные:")
    print(f"X* = {x_star}")
```

```
    print()

    print("Таблица значений функции:")
    print("i        x_i        y_i")
    print("--------------------------")
    for i in range(len(xs)):
        print(f"{i:<8} {xs[i]:<10.4f} {ys[i]:<10.4f}")

    print()

    is_uniform, h = check_uniform_grid(xs)

    if is_uniform:
        print(f"Проверка сетки: сетка равномерная, h = {h:.4f}")
    else:
        print("Проверка сетки: сетка неравномерная")

    print()

    result = calculate_derivatives(xs, ys, x_star)

    print("Для вычислений выбраны три соседние точки:")
    print(f"x_i     = {result['x_i']:.4f}, y_i     = {result['y_i']:.4f}")
    print(f"x_i+1   = {result['x_i1']:.4f}, y_i+1   = {result['y_i1']:.4f}")
    print(f"x_i+2   = {result['x_i2']:.4f}, y_i+2   = {result['y_i2']:.4f}")
    print()

    print("Вычисление по формуле (3.18):")
    print(
        "Левосторонняя производная на отрезке "
        f"[{result['x_i']:.4f}; {result['x_i1']:.4f}]: "
        f"{result['left_derivative']:.6f}"
    )
    print(
        "Правосторонняя производная на отрезке "
        f"[{result['x_i1']:.4f}; {result['x_i2']:.4f}]: "
        f"{result['right_derivative']:.6f}"
    )
    print()

    print("Вычисление первой производной со вторым порядком точности по
    ↪  формуле (3.20):")
    print(f"y'({x_star}) ≈ {result['first_derivative_second_order']:.6f}")
    print()

    print("Вычисление второй производной по формуле (3.21):")
    print(f"y''({x_star}) ≈ {result['second_derivative']:.6f}")


if __name__ == "__main__":
    main()
```

Вывод программы:

```
Лабораторная работа 3.4
Тема: численное дифференцирование
Вариант: 24

Исходные данные:
```

```
X* = 1.4

Таблица значений функции:
i        x_i        y_i
--------------------------
0        1.0000     2.0000
1        1.2000     2.1344
2        1.4000     2.4702
3        1.6000     2.9506
4        1.8000     3.5486

Проверка сетки: сетка равномерная, h = 0.2000

Для вычислений выбраны три соседние точки:
x_i     = 1.2000, y_i     = 2.1344
x_i+1   = 1.4000, y_i+1   = 2.4702
x_i+2   = 1.6000, y_i+2   = 2.9506

Вычисление по формуле (3.18):
Левосторонняя производная на отрезке [1.2000; 1.4000]: 1.679000
Правосторонняя производная на отрезке [1.4000; 1.6000]: 2.402000

Вычисление первой производной со вторым порядком точности по формуле (3.20):
y'(1.4) ≈ 2.040500

Вычисление второй производной по формуле (3.21):
y''(1.4) ≈ 3.615000
```

3.4.5. Анализ результатов

При $h = 0.2$ вычислены значения

$$y'(1.4) \approx 2.040500, \qquad y''(1.4) \approx 3.615000.$$

3.5. Задание 3.5

3.5.1. Формулировка задания

Вычислить определённый интеграл методами прямоугольников, трапеций и Симпсона на двух сетках. Уточнить результат методом Рунге--Ромберга.

3.5.2. Вариант № 24

$$I = \int_{-2}^{2} \sqrt{16 - x^2}\, dx, \qquad h_1 = 1, \qquad h_2 = 0.5.$$

3.5.3. Основные формулы

Составные квадратурные формулы на равномерной сетке:

$$I_{\text{пр}} = h \sum_{i=0}^{N-1} f\left(\frac{x_i + x_{i+1}}{2}\right),$$

$$I_{\text{тр}} = h \left(\frac{f_0 + f_N}{2} + \sum_{i=1}^{N-1} f_i\right),$$

$$I_{\mathrm{C}} = \frac{h}{3}\left(f_0 + f_N + 4 \sum_{\text{нечётн. } i} f_i + 2 \sum_{\substack{\text{чётн. } i \\ i \neq 0, N}} f_i \right).$$

Уточнение Рунге--Ромберга:

$$I_{\mathrm{RR}} = I_{h_2} + \frac{I_{h_2} - I_{h_1}}{2^p - 1}.$$

3.5.4. Текст программы

```
import math


# ============================================================
# Лабораторная работа 3.5
# Численное интегрирование
# Вариант 24
#
# Нужно вычислить:
#     integral from -2 to 2 of sqrt(16 - x^2) dx
#
# Нужно:
# 1) метод прямоугольников;
# 2) метод трапеций;
# 3) метод Симпсона;
# 4) оценку/уточнение по методу Рунге-Ромберга.
# ============================================================


def f(x):
    """
    Подынтегральная функция варианта 24:
        y = sqrt(16 - x^2)
    """
    return math.sqrt(16 - x ** 2)


def build_grid(x0, xk, h):
    """
    Строим равномерную сетку x_i = x0 + i*h.

    В методичке для формул численного интегрирования используется
    постоянный шаг h, поэтому проверяем, что отрезок [x0, xk]
    делится на целое число частей.
    """
    n_float = (xk - x0) / h
    n = int(round(n_float))

    # Техническая проверка корректности шага:
    # количество интервалов должно быть целым.
    if abs(n_float - n) > 1e-12:
        raise ValueError("Шаг h не делит отрезок [X0, Xk] на целое число
        ↪  частей")

    return [x0 + i * h for i in range(n + 1)]


def rectangle_method(x0, xk, h):
    """
```

```
    Метод прямоугольников по формуле:

        F ≈ h * sum f((x_{i-1} + x_i) / 2)

    То есть на каждом отрезке [x_{i-1}, x_i]
    функция заменяется значением в середине отрезка.
    """
    x = build_grid(x0, xk, h)
    n = len(x) - 1

    result = 0.0

    # Идем по всем частичным отрезкам [x_{i-1}, x_i]
    for i in range(1, n + 1):
        middle = (x[i - 1] + x[i]) / 2
        result += h * f(middle)

    return result


def trapezoid_method(x0, xk, h):
    """
    Метод трапеций по формуле:

        F ≈ h * (f_0 / 2 + f_1 + ... + f_{N-1} + f_N / 2)

    То есть на каждом отрезке [x_{i-1}, x_i]
    функция заменяется прямой, проходящей через концы отрезка.
    """
    x = build_grid(x0, xk, h)
    n = len(x) - 1

    result = f(x[0]) / 2 + f(x[n]) / 2

    # Внутренние узлы входят с коэффициентом 1
    for i in range(1, n):
        result += f(x[i])

    result *= h
    return result


def simpson_method(x0, xk, h):
    """
    Метод Симпсона по формуле:

        F ≈ h/3 * (f_0 + 4f_1 + 2f_2 + 4f_3 + ... + 2f_{N-2} + 4f_{N-1} + f_N)

    Важно:
    метод Симпсона требует четное количество интервалов,
    потому что парабола строится на каждой паре соседних отрезков.
    """
    x = build_grid(x0, xk, h)
    n = len(x) - 1

    # Проверка из условия применимости метода Симпсона:
    # число интервалов должно быть четным.
    if n % 2 != 0:
        raise ValueError("Для метода Симпсона число интервалов должно быть
        ↪  четным")
```

```
    result = f(x[0]) + f(x[n])

    # Внутренние узлы:
    # нечетные индексы имеют коэффициент 4,
    # четные индексы имеют коэффициент 2.
    for i in range(1, n):
        if i % 2 == 1:
            result += 4 * f(x[i])
        else:
            result += 2 * f(x[i])

    result *= h / 3
    return result


def runge_romberg(F_h, F_kh, p, k):
    """
    Метод Рунге-Ромберга по формуле методички:

        F ≈ F_h + (F_h - F_kh) / (k^p - 1)

    Здесь:
        F_h  — значение на более мелкой сетке;
        F_kh — значение на более крупной сетке;
        k    — во сколько раз крупный шаг больше мелкого;
        p    — порядок точности метода.

    Для нашей работы:
        k = h1 / h2 = 1.0 / 0.5 = 2

    Порядки точности:
        p = 2 для метода прямоугольников;
        p = 2 для метода трапеций;
        p = 4 для метода Симпсона.
    """
    return F_h + (F_h - F_kh) / (k ** p - 1)


def exact_integral_for_check():
    """
    Точное значение интеграла используем только для проверки результата.

    integral sqrt(16 - x^2) dx =
        x/2 * sqrt(16 - x^2) + 8 * arcsin(x / 4)

    На отрезке [-2, 2]:
        F_exact = 4*sqrt(3) + 8*pi/3
    """
    return 4 * math.sqrt(3) + 8 * math.pi / 3


def print_table_for_step(x0, xk, h):
    """
    Печатаем таблицу, как таблицу из примера методички:
    x_i, y_i и накопленные значения интеграла от X0 до x_i
    для трех методов.

    Для метода Симпсона значение печатается только в четных узлах,
    потому что формула Симпсона работает на парах интервалов.
```

```
    """
    x = build_grid(x0, xk, h)
    n = len(x) - 1

    print()
    print(f"Таблица вычислений для h = {h}")
    print("-" * 86)
    print(f"{'i':>2} {'x_i':>12} {'y_i':>14} {'прямоуг.':>14} {'трапеций':>14}
    ↪  {'Симпсона':>14}")
    print("-" * 86)

    rect_sum = 0.0
    trap_sum = 0.0

    for i in range(n + 1):
        xi = x[i]
        yi = f(xi)

        # В нулевом узле интеграл от X0 до X0 равен 0.
        if i == 0:
            rect_str = f"{0.0:.8f}"
            trap_str = f"{0.0:.8f}"
            simp_str = f"{0.0:.8f}"
        else:
            # Добавляем вклад текущего отрезка [x_{i-1}, x_i]
            middle = (x[i - 1] + x[i]) / 2
            rect_sum += h * f(middle)
            trap_sum += h * (f(x[i - 1]) + f(x[i])) / 2

            rect_str = f"{rect_sum:.8f}"
            trap_str = f"{trap_sum:.8f}"

            # Симпсон считается только после каждой пары интервалов.
            if i % 2 == 0:
                simp_value = simpson_method(x0, xi, h)
                simp_str = f"{simp_value:.8f}"
            else:
                simp_str = "—"

        print(f"{i:>2} {xi:>12.6f} {yi:>14.8f} {rect_str:>14} {trap_str:>14}
        ↪  {simp_str:>14}")

    print("-" * 86)


def main():
    # Исходные данные варианта 24
    x0 = -2.0
    xk = 2.0
    h1 = 1.0
    h2 = 0.5

    # Проверяем, что для метода Симпсона количество интервалов четное
    # при обоих шагах.
    for h in (h1, h2):
        n = int(round((xk - x0) / h))
        if n % 2 != 0:
            raise ValueError(f"При h = {h} число интервалов N = {n}, оно
            ↪  нечетное")
```

```
print("ЛР 3.5, вариант 24")
print("Интеграл: integral from -2 to 2 of sqrt(16 - x^2) dx")
print(f"X0 = {x0}, Xk = {xk}, h1 = {h1}, h2 = {h2}")

# Таблицы для двух шагов, как в примере методички
print_table_for_step(x0, xk, h1)
print_table_for_step(x0, xk, h2)

# Вычисления тремя методами на крупном и мелком шагах
rect_h1 = rectangle_method(x0, xk, h1)
rect_h2 = rectangle_method(x0, xk, h2)

trap_h1 = trapezoid_method(x0, xk, h1)
trap_h2 = trapezoid_method(x0, xk, h2)

simp_h1 = simpson_method(x0, xk, h1)
simp_h2 = simpson_method(x0, xk, h2)

# В методе Рунге-Ромберга:
# F_h  — значение на мелком шаге h2,
# F_kh — значение на крупном шаге h1,
# k = h1 / h2.
k = h1 / h2

rect_rr = runge_romberg(rect_h2, rect_h1, p=2, k=k)
trap_rr = runge_romberg(trap_h2, trap_h1, p=2, k=k)
simp_rr = runge_romberg(simp_h2, simp_h1, p=4, k=k)

# Оценка погрешности по Рунге-Ромбергу:
# это модуль поправки, то есть |F_RR - F_h|.
rect_error_estimate = abs(rect_rr - rect_h2)
trap_error_estimate = abs(trap_rr - trap_h2)
simp_error_estimate = abs(simp_rr - simp_h2)

exact = exact_integral_for_check()

print()
print("Итоговые значения интеграла")
print("-" * 78)
print(f"{'Метод':<22} {'h1 = 1.0':>14} {'h2 = 0.5':>14}
 ↪  {'Рунге-Ромберг':>18}")
print("-" * 78)
print(f"{'Прямоугольников':<22} {rect_h1:>14.8f} {rect_h2:>14.8f}
 ↪  {rect_rr:>18.8f}")
print(f"{'Трапеций':<22} {trap_h1:>14.8f} {trap_h2:>14.8f}
 ↪  {trap_rr:>18.8f}")
print(f"{'Симпсона':<22} {simp_h1:>14.8f} {simp_h2:>14.8f}
 ↪  {simp_rr:>18.8f}")
print("-" * 78)

print()
print("Оценка погрешности по Рунге-Ромбергу")
print("-" * 78)
print(f"{'Метод':<22} {'|F_RR - F_h2|':>20}")
print("-" * 78)
print(f"{'Прямоугольников':<22} {rect_error_estimate:>20.8f}")
print(f"{'Трапеций':<22} {trap_error_estimate:>20.8f}")
print(f"{'Симпсона':<22} {simp_error_estimate:>20.8f}")
print("-" * 78)
```

```
    print()
    print("Проверка по точному значению")
    print("-" * 78)
    print(f"Точное значение: {exact:.8f}")
    print(f"Абсолютная ошибка метода прямоугольников после Рунге-Ромберга:
     ↪  {abs(rect_rr - exact):.8f}")
    print(f"Абсолютная ошибка метода трапеций после Рунге-Ромберга:
     ↪  {abs(trap_rr - exact):.8f}")
    print(f"Абсолютная ошибка метода Симпсона после Рунге-Ромберга:
     ↪  {abs(simp_rr - exact):.8f}")
    print("-" * 78)


if __name__ == "__main__":
    main()
```

Вывод программы:

```
ЛР 3.5, вариант 24
Интеграл: integral from -2 to 2 of sqrt(16 - x^2) dx
X0 = -2.0, Xk = 2.0, h1 = 1.0, h2 = 0.5

Таблица вычислений для h = 1.0
------------------------------------------------------------------------------
 ↪  --------
 i          x_i            y_i       прямоуг.       трапеций       Симпсона
------------------------------------------------------------------------------
 ↪  --------
 0    -2.000000     3.46410162     0.00000000     0.00000000     0.00000000
 1    -1.000000     3.87298335     3.70809924     3.66854248              —
 2     0.000000     4.00000000     7.67672621     7.60503415     7.65201167
 3     1.000000     3.87298335    11.64535318    11.54152583              —
 4     2.000000     3.46410162    15.35345242    15.21006831    15.30402333
------------------------------------------------------------------------------
 ↪  --------

Таблица вычислений для h = 0.5
------------------------------------------------------------------------------
 ↪  --------
 i          x_i            y_i       прямоуг.       трапеций       Симпсона
------------------------------------------------------------------------------
 ↪  --------
 0    -2.000000     3.46410162     0.00000000     0.00000000     0.00000000
 1    -1.500000     3.70809924     1.79843682     1.79305021              —
 2    -1.000000     3.87298335     3.69827234     3.68832086     3.69491366
 3    -0.500000     3.96862697     5.66280155     5.64872344              —
 4     0.000000     4.00000000     7.65889147     7.64088018     7.65282886
 5     0.500000     3.96862697     9.65498140     9.63303692              —
 6     1.000000     3.87298335    11.61951061    11.59343950    11.61074406
 7     1.500000     3.70809924    13.51934613    13.48871015              —
 8     2.000000     3.46410162    15.31778295    15.28176036    15.30565772
------------------------------------------------------------------------------
 ↪  --------

Итоговые значения интеграла
------------------------------------------------------------------------------
Метод                        h1 = 1.0       h2 = 0.5      Рунге-Ромберг
------------------------------------------------------------------------------
Прямоугольников           15.35345242    15.31778295        15.30589312
```

```
Трапеций                  15.21006831    15.28176036        15.30565772
Симпсона                  15.30402333    15.30565772        15.30576667
-------------------------------------------------------------------------

Оценка погрешности по Рунге-Ромбергу
-------------------------------------------------------------------------
Метод                          |F_RR - F_h2|
-------------------------------------------------------------------------
Прямоугольников                  0.01188982
Трапеций                         0.02389735
Симпсона                         0.00010896
-------------------------------------------------------------------------

Проверка по точному значению
-------------------------------------------------------------------------
Точное значение: 15.30578364
Абсолютная ошибка метода прямоугольников после Рунге-Ромберга: 0.00010948
Абсолютная ошибка метода трапеций после Рунге-Ромберга:       0.00012592
Абсолютная ошибка метода Симпсона после Рунге-Ромберга:       0.00001696
-------------------------------------------------------------------------
```

3.5.5. Анализ результатов

Точное значение равно 15.30578364. После уточнения Рунге--Ромберга наименьшую абсолютную ошибку показала формула Симпсона:

$$|I - I_{\text{RR}}| = 0.00001696.$$

3.6. Выводы

Реализованы методы интерполяции, сплайн-аппроксимации, наименьших квадратов, численного дифференцирования и интегрирования. Эквивалентные формы интерполяционного многочлена дали одинаковые значения. В задаче МНК квадратичный многочлен лучше линейного по сумме квадратов невязок. При вычислении интеграла наиболее точным оказался метод Симпсона с уточнением.

4. Решение дифференциальных уравнений

4.1. Задание 4.1

4.1.1. Формулировка задания

Решить задачу Коши для обыкновенного дифференциального уравнения второго порядка методами Эйлера, Рунге--Кутты и Адамса. Оценить погрешность методом Рунге--Ромберга и сравнением с точным решением.

4.1.2. Вариант № 24

$$x^2y'' + (x+1)y' - y = 0, \qquad y(1) = 2 + e, \quad y'(1) = 1, \quad x \in [1, 2], \quad h = 0.1.$$

Точное решение:

$$y(x) = x + 1 + xe^{1/x}.$$

4.1.3. Основные формулы

После замены $z = y'$ задача сводится к системе

$$y' = z, \qquad z' = \frac{y - (x+1)z}{x^2}.$$

Явный метод Эйлера для вектора $Y = (y, z)^T$:

$$Y_{k+1} = Y_k + hF(x_k, Y_k).$$

Метод Рунге--Кутты четвёртого порядка:

$$Y_{k+1} = Y_k + \frac{K_1 + 2K_2 + 2K_3 + K_4}{6}.$$

Для оценки ошибки решения порядка p используется формула

$$R_{\text{RR}} = \frac{|y_{h/2} - y_h|}{2^p - 1}.$$

4.1.4. Текст программы

```
import math

# ------------------------------------------------------------
# Лабораторная работа 4.1
# Вариант 24
# Численные методы решения задачи Коши для ОДУ 2-го порядка:
#
#     x^2 * y'' + (x + 1) * y' - y = 0
#     y(1) = 2 + e
#     y'(1) = 1
#     x in [1, 2], h = 0.1
#
# Точное решение:
#     y(x) = x + 1 + x * e^(1/x)
#
# По методичке ОДУ 2-го порядка сводим к системе двух ОДУ 1-го порядка:
#     z = y'
#     y' = z
#     z' = (y - (x + 1) * z) / x^2
# ------------------------------------------------------------


def exact_y(x):
    """Точное решение y(x) для оценки погрешности."""
    return x + 1.0 + x * math.exp(1.0 / x)


def exact_z(x):
    """Точная производная y'(x), нужна только для контроля z."""
    return 1.0 + math.exp(1.0 / x) - math.exp(1.0 / x) / x


def f(x, y, z):
    """Правая часть первого уравнения системы: y' = z."""
    return z
```

```
def g(x, y, z):
    """Правая часть второго уравнения системы: z' = y''."""
    return (y - (x + 1.0) * z) / (x * x)


def euler_method(a, b, h, y0, z0):
    """
    Явный метод Эйлера для системы двух ОДУ 1-го порядка.

    По методичке для одного уравнения:
        y_{k+1} = y_k + h * f(x_k, y_k)

    Для системы применяем ту же формулу к каждой компоненте:
        y_{k+1} = y_k + h * f(x_k, y_k, z_k)
        z_{k+1} = z_k + h * g(x_k, y_k, z_k)
    """
    n = int(round((b - a) / h))
    x_values = [a]
    y_values = [y0]
    z_values = [z0]

    for k in range(n):
        xk = x_values[k]
        yk = y_values[k]
        zk = z_values[k]

        # Сначала считаем приращения строго из левой точки интервала.
        dy = h * f(xk, yk, zk)
        dz = h * g(xk, yk, zk)

        # Затем переходим в следующий узел сетки.
        x_values.append(xk + h)
        y_values.append(yk + dy)
        z_values.append(zk + dz)

    return x_values, y_values, z_values


def rk4_step(xk, yk, zk, h):
    """
    Один шаг метода Рунге-Кутты 4-го порядка для системы.

    Обозначения соответствуют формулам методички для системы:
        K_i — приращения для y,
        L_i — приращения для z.
    """
    K1 = h * f(xk, yk, zk)
    L1 = h * g(xk, yk, zk)

    K2 = h * f(xk + h / 2.0, yk + K1 / 2.0, zk + L1 / 2.0)
    L2 = h * g(xk + h / 2.0, yk + K1 / 2.0, zk + L1 / 2.0)

    K3 = h * f(xk + h / 2.0, yk + K2 / 2.0, zk + L2 / 2.0)
    L3 = h * g(xk + h / 2.0, yk + K2 / 2.0, zk + L2 / 2.0)

    K4 = h * f(xk + h, yk + K3, zk + L3)
    L4 = h * g(xk + h, yk + K3, zk + L3)

    dy = (K1 + 2.0 * K2 + 2.0 * K3 + K4) / 6.0
    dz = (L1 + 2.0 * L2 + 2.0 * L3 + L4) / 6.0
```

```
    theta_y = None
    theta_z = None

    # Контроль правильности выбора шага h по методичке для РК4:
    # theta_1 = (K3 - K2) / (K2 - K1), theta_2 = (L3 - L2) / (L2 - L1).
    # Если знаменатель близок к нулю, параметр не считаем, чтобы не делить на
    ↪ ноль.
    if abs(K2 - K1) > 1e-15:
        theta_y = abs((K3 - K2) / (K2 - K1))
    if abs(L2 - L1) > 1e-15:
        theta_z = abs((L3 - L2) / (L2 - L1))

    return dy, dz, theta_y, theta_z


def runge_kutta_4_method(a, b, h, y0, z0):
    """Метод Рунге-Кутты 4-го порядка для системы двух ОДУ."""
    n = int(round((b - a) / h))
    x_values = [a]
    y_values = [y0]
    z_values = [z0]
    theta_values = []

    for k in range(n):
        xk = x_values[k]
        yk = y_values[k]
        zk = z_values[k]

        dy, dz, theta_y, theta_z = rk4_step(xk, yk, zk, h)

        x_values.append(xk + h)
        y_values.append(yk + dy)
        z_values.append(zk + dz)
        theta_values.append((theta_y, theta_z))

    return x_values, y_values, z_values, theta_values


def adams_4_method(a, b, h, y0, z0):
    """
    Явный метод Адамса 4-го порядка для системы.

    Метод Адамса не является самостартующим, поэтому первые 4 узла
    x0, x1, x2, x3 получаем методом Рунге-Кутты 4-го порядка.

    Формула Адамса 4-го порядка из методички:
        y_{k+1} = y_k + h/24 * (55 f_k - 59 f_{k-1} + 37 f_{k-2} - 9 f_{k-3})

    Для системы отдельно применяем ее к y и z:
        для y используем f = y' = z,
        для z используем g = z'.
    """
    n = int(round((b - a) / h))

    if n < 4:
        raise ValueError("Для метода Адамса 4-го порядка нужно минимум 4 шага
        ↪ сетки.")

    x_values = [a]
```

```
    y_values = [y0]
    z_values = [z0]

    # Стартовые значения y1, y2, y3 получаем методом Рунге-Кутты 4-го порядка.
    for k in range(3):
        xk = x_values[k]
        yk = y_values[k]
        zk = z_values[k]

        dy, dz, theta_y, theta_z = rk4_step(xk, yk, zk, h)

        x_values.append(xk + h)
        y_values.append(yk + dy)
        z_values.append(zk + dz)

    # Далее применяем многошаговую формулу Адамса.
    for k in range(3, n):
        xk = x_values[k]
        yk = y_values[k]
        zk = z_values[k]

        f_k = f(x_values[k], y_values[k], z_values[k])
        f_k1 = f(x_values[k - 1], y_values[k - 1], z_values[k - 1])
        f_k2 = f(x_values[k - 2], y_values[k - 2], z_values[k - 2])
        f_k3 = f(x_values[k - 3], y_values[k - 3], z_values[k - 3])

        g_k = g(x_values[k], y_values[k], z_values[k])
        g_k1 = g(x_values[k - 1], y_values[k - 1], z_values[k - 1])
        g_k2 = g(x_values[k - 2], y_values[k - 2], z_values[k - 2])
        g_k3 = g(x_values[k - 3], y_values[k - 3], z_values[k - 3])

        dy = h / 24.0 * (55.0 * f_k - 59.0 * f_k1 + 37.0 * f_k2 - 9.0 * f_k3)
        dz = h / 24.0 * (55.0 * g_k - 59.0 * g_k1 + 37.0 * g_k2 - 9.0 * g_k3)

        x_values.append(xk + h)
        y_values.append(yk + dy)
        z_values.append(zk + dz)

    return x_values, y_values, z_values


def runge_romberg_errors(y_h, y_h2, p):
    """
    Оценка погрешности по методу Рунге-Ромберга.

    y_h  — решение на сетке с шагом h,
    y_h2 — решение на сетке с шагом h/2,
    p    — порядок точности метода.

    Сравниваем значения в одинаковых узлах:
        x_k на крупной сетке соответствует x_{2k} на мелкой сетке.

    Оценка главного члена погрешности:
        R = |(y_{h/2} - y_h) / (2^p - 1)|
    """
    result = []
    denominator = 2.0 ** p - 1.0

    for k in range(len(y_h)):
        result.append(abs((y_h2[2 * k] - y_h[k]) / denominator))
```

```
    return result


def print_table(method_name, x_values, y_values, z_values, rr_errors=None,
↪  theta_values=None):
    """Печать итоговой таблицы для выбранного метода."""
    print("\n" + method_name)
    print("-" * 118)

    if rr_errors is None:
        print(f"{'k':>2} {'x_k':>8} {'y_k':>16} {'z_k':>16} {'y точн.':>16}
        ↪  {'|y-yточн|':>16}")
    else:
        print(f"{'k':>2} {'x_k':>8} {'y_k':>16} {'z_k':>16} {'y точн.':>16}
        ↪  {'|y-yточн|':>16} {'R_R':>16}")

    for k in range(len(x_values)):
        xk = x_values[k]
        y_exact = exact_y(xk)
        exact_error = abs(y_values[k] - y_exact)

        if rr_errors is None:
            print(f"{k:2d} {xk:8.4f} {y_values[k]:16.9f} {z_values[k]:16.9f}
            ↪  {y_exact:16.9f} {exact_error:16.9e}")
        else:
            print(f"{k:2d} {xk:8.4f} {y_values[k]:16.9f} {z_values[k]:16.9f}
            ↪  {y_exact:16.9f} {exact_error:16.9e} {rr_errors[k]:16.9e}")

    if theta_values is not None:
        print("\nКонтроль шага для метода Рунге-Кутты 4-го порядка:")
        print(f"{'k':>2} {'theta_y':>16} {'theta_z':>16} {'вывод':>35}")
        for k in range(len(theta_values)):
            theta_y, theta_z = theta_values[k]

            # По методичке: несколько сотых — нормально, >0.1 — шаг уменьшить,
            ↪  <0.01 — можно увеличить.
            max_theta = 0.0
            if theta_y is not None:
                max_theta = max(max_theta, theta_y)
            if theta_z is not None:
                max_theta = max(max_theta, theta_z)

            if max_theta > 0.1:
                conclusion = "шаг лучше уменьшить"
            elif max_theta < 0.01:
                conclusion = "шаг можно увеличить"
            else:
                conclusion = "шаг приемлем"

            theta_y_text = "-" if theta_y is None else f"{theta_y:.9f}"
            theta_z_text = "-" if theta_z is None else f"{theta_z:.9f}"
            print(f"{k:2d} {theta_y_text:>16} {theta_z_text:>16}
            ↪  {conclusion:>35}")


def print_max_errors(method_name, x_values, y_values, rr_errors):
    """Печать максимальной ошибки по точному решению и по Рунге-Ромбергу."""
    exact_errors = []
    for k in range(len(x_values)):
```

```
        exact_errors.append(abs(y_values[k] - exact_y(x_values[k])))

    print(f"\n{method_name}")
    print(f"Максимальная погрешность по точному решению:
    ↪  {max(exact_errors):.9e}")
    print(f"Максимальная оценка по Рунге-Ромбергу:     {max(rr_errors):.9e}")


def main():
    # Исходные данные варианта 24.
    a = 1.0
    b = 2.0
    h = 0.1
    h2 = h / 2.0
    y0 = 2.0 + math.e
    z0 = 1.0

    print("Лабораторная работа 4.1. Вариант 24")
    print("ОДУ: x^2*y'' + (x + 1)*y' - y = 0")
    print("Начальные условия: y(1)=2+e, y'(1)=1")
    print("Отрезок: [1, 2]")
    print("Основной шаг h =", h)
    print("Шаг для Рунге-Ромберга h/2 =", h2)
    print("Точное решение: y = x + 1 + x*e^(1/x)")

    # Решения с основным шагом h.
    x_e, y_e, z_e = euler_method(a, b, h, y0, z0)
    x_rk, y_rk, z_rk, theta_rk = runge_kutta_4_method(a, b, h, y0, z0)
    x_ad, y_ad, z_ad = adams_4_method(a, b, h, y0, z0)

    # Решения с половинным шагом h/2 для оценки Рунге-Ромберга.
    x_e2, y_e2, z_e2 = euler_method(a, b, h2, y0, z0)
    x_rk2, y_rk2, z_rk2, theta_rk2 = runge_kutta_4_method(a, b, h2, y0, z0)
    x_ad2, y_ad2, z_ad2 = adams_4_method(a, b, h2, y0, z0)

    # Порядки точности методов.
    rr_e = runge_romberg_errors(y_e, y_e2, p=1)
    rr_rk = runge_romberg_errors(y_rk, y_rk2, p=4)
    rr_ad = runge_romberg_errors(y_ad, y_ad2, p=4)

    print_table("Метод Эйлера", x_e, y_e, z_e, rr_e)
    print_table("Метод Рунге-Кутты 4-го порядка", x_rk, y_rk, z_rk, rr_rk,
    ↪  theta_rk)
    print_table("Метод Адамса 4-го порядка", x_ad, y_ad, z_ad, rr_ad)

    print("\nИтоговые максимальные погрешности")
    print("=" * 60)
    print_max_errors("Метод Эйлера", x_e, y_e, rr_e)
    print_max_errors("Метод Рунге-Кутты 4-го порядка", x_rk, y_rk, rr_rk)
    print_max_errors("Метод Адамса 4-го порядка", x_ad, y_ad, rr_ad)


if __name__ == "__main__":
    main()
```

Вывод программы:

```
Лабораторная работа 4.1. Вариант 24
ОДУ: x^2*y'' + (x + 1)*y' - y = 0
```

```
Начальные условия: y(1)=2+e, y'(1)=1
Отрезок: [1, 2]
Основной шаг h = 0.1
Шаг для Рунге-Ромберга h/2 = 0.05
Точное решение: y = x + 1 + x*e^(1/x)

Метод Эйлера
-------------------------------------------------------------------------------
 ↪  ----------------------------------------
 k      x_k             y_k             z_k          y точн.        |y-уточн|
  ↪             R_R
 0   1.0000      4.718281828      1.000000000      4.718281828  0.000000000e+00
  ↪   0.000000000e+00
 1   1.1000      4.818281828      1.271828183      4.830271593  1.198976463e-02
  ↪   6.795704571e-03
 2   1.2000      4.945464647      1.449302782      4.961171069  1.570642233e-02
  ↪   8.829060732e-03
 3   1.3000      5.090394925      1.571316568      5.105537194  1.514226923e-02
  ↪   8.480569355e-03
 4   1.4000      5.247526582      1.658675552      5.259817898  1.229131665e-02
  ↪   6.901042482e-03
 5   1.5000      5.413394137      1.723303371      5.421601062  8.206924714e-03
  ↪   4.680981855e-03
 6   1.6000      5.585724474      1.772420513      5.589193532  3.469057944e-03
  ↪   2.136797816e-03
 7   1.7000      5.762966525      1.810601417      5.761373113  1.593411907e-03
  ↪   5.568230147e-04
 8   1.8000      5.944026667      1.840855490      5.937236198  6.790469484e-03
  ↪   3.301744126e-03
 9   1.9000      6.128112216      1.865226826      6.116100741  1.201147540e-02
  ↪   6.042614909e-03
10   2.0000      6.314634899      1.885142461      6.297442541  1.719235724e-02
 ↪   8.748632934e-03

Метод Рунге-Кутты 4-го порядка
-------------------------------------------------------------------------------
 ↪  ----------------------------------------
 k      x_k             y_k             z_k          y точн.        |y-уточн|
  ↪             R_R
 0   1.0000      4.718281828      1.000000000      4.718281828  0.000000000e+00
  ↪   0.000000000e+00
 1   1.1000      4.830277029      1.225639069      4.830271593  5.435553301e-06
  ↪   3.410784312e-07
 2   1.2000      4.961178895      1.383492329      4.961171069  7.826075032e-06
  ↪   4.909871348e-07
 3   1.3000      5.105546110      1.498021082      5.105537194  8.916309584e-06
  ↪   5.593030994e-07
 4   1.4000      5.259827318      1.583633616      5.259817898  9.419549530e-06
  ↪   5.908023296e-07
 5   1.5000      5.421610711      1.649242572      5.421601062  9.649518640e-06
  ↪   6.051701463e-07
 6   1.6000      5.589203284      1.700590646      5.589193532  9.752129605e-06
  ↪   6.115586547e-07
 7   1.7000      5.761382911      1.741507919      5.761373113  9.798071125e-06
  ↪   6.144000047e-07
 8   1.8000      5.937246020      1.774625463      5.937236198  9.822492930e-06
  ↪   6.158971830e-07
 9   1.9000      6.116110584      1.801797533      6.116100741  9.843038836e-06
  ↪   6.171554763e-07
10   2.0000      6.297452410      1.824360473      6.297442541  9.868449371e-06
 ↪   6.187220725e-07
```

```
Контроль шага для метода Рунге-Кутты 4-го порядка:
 k          theta_y          theta_z                                  вывод
 0      0.185941043      0.105165643                  шаг лучше уменьшить
 1      0.166351607      0.092649081                  шаг лучше уменьшить
 2      0.150400000      0.082638298                  шаг лучше уменьшить
 3      0.137174211      0.074471879                  шаг лучше уменьшить
 4      0.126040428      0.067697934                  шаг лучше уменьшить
 5      0.116545265      0.061998290                  шаг лучше уменьшить
 6      0.108356290      0.057143079                  шаг лучше уменьшить
 7      0.101224490      0.052962475                  шаг лучше уменьшить
 8      0.094959825      0.049328539                         шаг приемлем
 9      0.089414859      0.046143211                         шаг приемлем

Метод Адамса 4-го порядка
--------------------------------------------------------------------------------
 ↪  ----------------------------------------
 k      x_k              y_k              z_k          y точн.        |y-уточн|
  ↪             R_R
 0   1.0000      4.718281828      1.000000000      4.718281828  0.000000000e+00
  ↪   0.000000000e+00
 1   1.1000      4.830277029      1.225639069      4.830271593  5.435553301e-06
  ↪   3.410784312e-07
 2   1.2000      4.961178895      1.383492329      4.961171069  7.826075032e-06
  ↪   8.560182152e-07
 3   1.3000      5.105546110      1.498021082      5.105537194  8.916309584e-06
  ↪   9.267545932e-07
 4   1.4000      5.260186767      1.581340547      5.259817898  3.688689211e-04
  ↪   2.414849114e-05
 5   1.5000      5.421640729      1.646500354      5.421601062  3.966735141e-05
  ↪   3.777387294e-06
 6   1.6000      5.589281464      1.697216784      5.589193532  8.793164306e-05
  ↪   8.741040650e-06
 7   1.7000      5.761076517      1.738303479      5.761373113  2.965968604e-04
  ↪   1.511331462e-05
 8   1.8000      5.936740472      1.771400083      5.937236198  4.957258255e-04
  ↪   2.663583913e-05
 9   1.9000      6.115263877      1.798767436      6.116100741  8.368640063e-04
  ↪   4.768083856e-05
10   2.0000      6.296355050      1.821430801      6.297442541  1.087491096e-03
 ↪   6.275629071e-05

Итоговые максимальные погрешности
============================================================

Метод Эйлера
Максимальная погрешность по точному решению: 1.719235724e-02
Максимальная оценка по Рунге-Ромбергу:     8.829060732e-03

Метод Рунге-Кутты 4-го порядка
Максимальная погрешность по точному решению: 9.868449371e-06
Максимальная оценка по Рунге-Ромбергу:     6.187220725e-07

Метод Адамса 4-го порядка
Максимальная погрешность по точному решению: 1.087491096e-03
Максимальная оценка по Рунге-Ромбергу:     6.275629071e-05
```

4.1.5. Анализ результатов

Максимальные ошибки по точному решению составили:

Метод	$\max \lvert y_{\text{точн}} - y_h \rvert$
Эйлера	$1.719235724 \cdot 10^{-2}$
Рунге--Кутты 4-го порядка	$9.868449371 \cdot 10^{-6}$
Адамса 4-го порядка	$1.087491096 \cdot 10^{-3}$

На выбранной сетке наибольшую точность обеспечил метод Рунге--Кутты четвёртого порядка.

4.2. Задание 4.2

4.2.1. Формулировка задания

Решить краевую задачу для ОДУ второго порядка методом стрельбы и конечно-разностным методом. Оценить погрешность методом Рунге--Ромберга и путём сравнения с точным решением.

4.2.2. Вариант № 24

$$(x^2+1)y'' - 2y = 0, \qquad y'(0) = 2, \quad y(1) = 3 + \frac{\pi}{2}, \quad h = 0.1.$$

Точное решение:

$$y(x) = x^2 + x + 1 + (x^2+1)\arctan x.$$

4.2.3. Основные формулы

В методе стрельбы неизвестный параметр $\eta = y(0)$ подбирается так, чтобы решение задачи Коши

$$y' = z, \qquad z' = \frac{2y}{x^2+1}, \qquad y(0) = \eta, \quad z(0) = 2$$

удовлетворяло правому граничному условию. Для функции невязки

$$\Phi(\eta) = y(1,\eta) - \left(3 + \frac{\pi}{2}\right)$$

применяется метод секущих.

В конечно-разностном методе во внутренних узлах используется аппроксимация

$$y''(x_k) \approx \frac{y_{k+1} - 2y_k + y_{k-1}}{h^2},$$

а производная на левой границе аппроксимируется односторонней разностью. Возникающая трёхдиагональная система решается методом прогонки.

4.2.4. Текст программы

```
import math

# Лабораторная работа 4.2, вариант 24
# Численные методы решения краевой задачи для ОДУ второго порядка.
# Требуется реализовать:
# 1) метод стрельбы;
```

```
# 2) конечно-разностный метод;
# 3) оценку погрешности по Рунге--Ромбергу;
# 4) сравнение с точным решением.

# ------------------------------------------------------------------------
↪  -
# Вариант 24:
#     (x^2 + 1) * y'' - 2y = 0,
#     y'(0) = 2,
#     y(1) = 3 + pi/2.
#
# Точное решение:
#     y(x) = x^2 + x + 1 + (x^2 + 1) * arctg(x).
#
# Для метода стрельбы сводим ОДУ второго порядка к системе первого порядка:
#     y' = z,
#     z' = 2y / (x^2 + 1).
# Так как на левом конце задано y'(0), а не y(0), неизвестным параметром
# стрельбы является eta = y(0). Подбираем eta методом секущих так, чтобы
# выполнялось правое граничное условие y(1) = 3 + pi/2.
# ------------------------------------------------------------------------
↪  -

A = 0.0
B = 1.0
Y_DERIV_A = 2.0
Y_B = 3.0 + math.pi / 2.0
EPS = 1e-6


def exact_solution(x):
    """Точное решение варианта 24."""
    return x * x + x + 1.0 + (x * x + 1.0) * math.atan(x)


def f_for_y(x, y, z):
    """Правая часть первого уравнения системы: y' = z."""
    return z


def f_for_z(x, y, z):
    """Правая часть второго уравнения системы: z' = 2y / (x^2 + 1)."""
    return 2.0 * y / (x * x + 1.0)


def runge_kutta_4_system(eta, h):
    """
    Решение задачи Коши методом Рунге-Кутты 4-го порядка по формулам методички.

    Начальные условия для текущего значения параметра стрельбы:
        y(0) = eta,
        z(0) = y'(0) = 2.

    Возвращает список точек (x_k, y_k, z_k).
    """
    n = int(round((B - A) / h))
    x = A
    y = eta
    z = Y_DERIV_A
```

```
    result = [(x, y, z)]

    for _ in range(n):
        # Формулы Рунге-Кутты 4-го порядка для системы:
        # K_i отвечают за приращение y, L_i отвечают за приращение z.
        k1 = h * f_for_y(x, y, z)
        l1 = h * f_for_z(x, y, z)

        k2 = h * f_for_y(x + h / 2.0, y + k1 / 2.0, z + l1 / 2.0)
        l2 = h * f_for_z(x + h / 2.0, y + k1 / 2.0, z + l1 / 2.0)

        k3 = h * f_for_y(x + h / 2.0, y + k2 / 2.0, z + l2 / 2.0)
        l3 = h * f_for_z(x + h / 2.0, y + k2 / 2.0, z + l2 / 2.0)

        k4 = h * f_for_y(x + h, y + k3, z + l3)
        l4 = h * f_for_z(x + h, y + k3, z + l3)

        delta_y = (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
        delta_z = (l1 + 2.0 * l2 + 2.0 * l3 + l4) / 6.0

        y = y + delta_y
        z = z + delta_z
        x = x + h

        # Округление x защищает от накопления вида 0.30000000000000004.
        result.append((round(x, 12), y, z))

    return result


def phi_for_shooting(eta, h):
    """
    Алгоритмическая функция Phi(eta) из метода стрельбы.
    Нужно добиться Phi(eta) = 0, то есть y(b, eta) - y_b = 0.
    """
    solution = runge_kutta_4_system(eta, h)
    y_at_b = solution[-1][1]
    return y_at_b - Y_B


def shooting_method(h, eta0=0.0, eta1=2.0, eps=EPS, max_iter=100):
    """
    Метод стрельбы + метод секущих.

    В методичке для параметра стрельбы используется итерация секущих:
        eta_{j+2} = eta_{j+1} - (eta_{j+1} - eta_j) * Phi(eta_{j+1}) /
                    (Phi(eta_{j+1}) - Phi(eta_j)).

    Критерий остановки:
        |Phi(eta_j)| <= eps,
    то есть правое граничное условие выполнено с заданной точностью.
    """
    phi0 = phi_for_shooting(eta0, h)
    phi1 = phi_for_shooting(eta1, h)

    iterations = [
        (0, eta0, runge_kutta_4_system(eta0, h)[-1][1], phi0),
        (1, eta1, runge_kutta_4_system(eta1, h)[-1][1], phi1),
    ]
```

```
    if abs(phi0) <= eps:
        return eta0, runge_kutta_4_system(eta0, h), iterations
    if abs(phi1) <= eps:
        return eta1, runge_kutta_4_system(eta1, h), iterations

    for j in range(max_iter):
        denominator = phi1 - phi0
        if abs(denominator) < 1e-14:
            raise ZeroDivisionError("Метод секущих остановлен: Phi(eta_j+1) -
            ↪  Phi(eta_j) слишком мало.")

        eta2 = eta1 - (eta1 - eta0) * phi1 / denominator
        phi2 = phi_for_shooting(eta2, h)
        y_at_b = runge_kutta_4_system(eta2, h)[-1][1]
        iterations.append((j + 2, eta2, y_at_b, phi2))

       # Проверка критерия остановки строго после получения нового приближения.
        if abs(phi2) <= eps:
            return eta2, runge_kutta_4_system(eta2, h), iterations

        eta0, phi0 = eta1, phi1
        eta1, phi1 = eta2, phi2

    raise RuntimeError("Метод стрельбы не сошелся за заданное число итераций.")


def p_coef(x):
    """Коэффициент p(x) в форме y'' + p(x)y' + q(x)y = f(x)."""
    return 0.0


def q_coef(x):
    """Коэффициент q(x) после деления исходного уравнения на x^2 + 1."""
    return -2.0 / (x * x + 1.0)


def right_side(x):
    """Правая часть f(x) после деления исходного уравнения на x^2 + 1."""
    return 0.0


def thomas_algorithm(a, b, c, d):
    """
    Метод прогонки для трехдиагональной системы.

    a[i] -- коэффициент при y_{i-1};
    b[i] -- коэффициент при y_i;
    c[i] -- коэффициент при y_{i+1};
    d[i] -- правая часть.
    """
    n = len(d)

    # Проверка размерностей массивов.
    if not (len(a) == len(b) == len(c) == n):
        raise ValueError("Размеры массивов коэффициентов трехдиагональной
        ↪  системы не совпадают.")

    # Прямой ход прогонки.
    alpha = [0.0] * n
    beta = [0.0] * n
```

```
    if abs(b[0]) < 1e-14:
        raise ZeroDivisionError("Нулевой первый диагональный элемент в методе
        ↪  прогонки.")

    alpha[0] = -c[0] / b[0]
    beta[0] = d[0] / b[0]

    for i in range(1, n):
        denominator = b[i] + a[i] * alpha[i - 1]
        if abs(denominator) < 1e-14:
            raise ZeroDivisionError("Нулевой знаменатель в прямом ходе метода
            ↪  прогонки.")

        alpha[i] = -c[i] / denominator if i < n - 1 else 0.0
        beta[i] = (d[i] - a[i] * beta[i - 1]) / denominator

    # Обратный ход прогонки.
    y = [0.0] * n
    y[n - 1] = beta[n - 1]

    for i in range(n - 2, -1, -1):
        y[i] = alpha[i] * y[i + 1] + beta[i]

    return y


def check_diagonal_dominance(lower, diag, upper):
    """
    Проверка преобладания диагональных элементов.

    В методичке отдельно отмечено, что при достаточно малых h и q(x_k)<0
    для трехдиагональной системы выполняется условие преобладания диагональных
    элементов, что гарантирует корректное применение метода прогонки.
    Здесь проверяем его численно для построенной системы.
    """
    for i in range(len(diag)):
        left = abs(lower[i])
        middle = abs(diag[i])
        right = abs(upper[i])
        if middle + 1e-14 < left + right:
            return False
    return True


def finite_difference_method(h):
    """
    Конечно-разностный метод.

    Методичка приводит центральные разности для внутренних узлов:
        y'_k  ≈ (y_{k+1} - y_{k-1}) / (2h),
        y''_k ≈ (y_{k+1} - 2y_k + y_{k-1}) / h^2.

    Для условия второго рода y'(0)=2 используем одностороннюю разность 1-го
    ↪  порядка
    из методички, чтобы сохранить трехдиагональную матрицу:
        (y_1 - y_0) / h = 2.

    Правое условие первого рода:
        y_N = 3 + pi/2.
```

```
    """
    n = int(round((B - A) / h))
    xs = [A + i * h for i in range(n + 1)]

    # Система строится для всех неизвестных y_0, y_1, ..., y_N.
    lower = [0.0] * (n + 1)
    diag = [0.0] * (n + 1)
    upper = [0.0] * (n + 1)
    rhs = [0.0] * (n + 1)

    # Левая граница: (y_1 - y_0) / h = 2  =>  -y_0 + y_1 = 2h.
    diag[0] = -1.0
    upper[0] = 1.0
    rhs[0] = Y_DERIV_A * h

    # Внутренние узлы: формула (4.38) после умножения на h^2:
    # (1 - p_k h/2)y_{k-1} + (-2 + h^2 q_k)y_k + (1 + p_k h/2)y_{k+1} = h^2 f_k.
    for k in range(1, n):
        xk = xs[k]
        pk = p_coef(xk)
        qk = q_coef(xk)
        fk = right_side(xk)

        lower[k] = 1.0 - pk * h / 2.0
        diag[k] = -2.0 + h * h * qk
        upper[k] = 1.0 + pk * h / 2.0
        rhs[k] = h * h * fk

    # Правая граница: y_N = 3 + pi/2.
    diag[n] = 1.0
    rhs[n] = Y_B

    # Проверка условия преобладания диагональных элементов перед прогонкой.
    # Для данной задачи q(x)<0, поэтому на внутренних узлах условие
    ↪ выполняется;
    # граничные строки также остаются корректными для метода прогонки.
    if not check_diagonal_dominance(lower, diag, upper):
        print("Предупреждение: диагональное преобладание не выполнено для h =",
        ↪ h)

    ys = thomas_algorithm(lower, diag, upper, rhs)
    return list(zip(xs, ys))


def runge_romberg_error(coarse_solution, fine_solution, order):
    """
    Оценка погрешности по Рунге--Ромбергу на узлах крупной сетки.

    coarse_solution -- решение с шагом h;
    fine_solution   -- решение с шагом h/2;
    order           -- порядок метода.

    На одинаковых узлах берем каждую вторую точку решения с шагом h/2.
    Оценка: |y_{h/2} - y_h| / (2^p - 1).
    """
    result = []
    denominator = 2.0 ** order - 1.0

    for i in range(len(coarse_solution)):
        x = coarse_solution[i][0]
```

```
        y_coarse = coarse_solution[i][1]
        y_fine = fine_solution[2 * i][1]
        error = abs(y_fine - y_coarse) / denominator
        result.append((x, error))

    return result


def print_iterations(iterations):
    print("\nИтерации метода секущих в методе стрельбы")
    print("j        eta_j              y(b, eta_j)        Phi(eta_j)")
    print("-" * 70)
    for j, eta, yb, phi in iterations:
        print(f"{j:<3d} {eta:>18.10f} {yb:>18.10f} {phi:>18.10e}")


def print_solution_table(title, solution, rr_errors=None):
    print("\n" + title)
    if rr_errors is None:
        print("k      x_k          y_k              y_exact
        ↪  |y_exact-y_k|")
        print("-" * 78)
        for k, (x, y) in enumerate(solution):
            y_exact = exact_solution(x)
            print(f"{k:<3d} {x:>8.4f} {y:>16.10f} {y_exact:>16.10f}
            ↪  {abs(y_exact - y):>16.10e}")
    else:
        print("k      x_k          y_k              y_exact          |exact-y|
        ↪      RR")
        print("-" * 95)
        for k, (x, y) in enumerate(solution):
            y_exact = exact_solution(x)
            rr = rr_errors[k][1]
            print(f"{k:<3d} {x:>8.4f} {y:>16.10f} {y_exact:>16.10f}
            ↪  {abs(y_exact - y):>16.10e} {rr:>16.10e}")


def main():
    h = 0.1
    h_half = h / 2.0

    print("Лабораторная работа 4.2. Вариант 24")
    print("Краевая задача: (x^2 + 1)y'' - 2y = 0")
    print("Граничные условия: y'(0)=2, y(1)=3+pi/2")
    print("Точное решение: y=x^2+x+1+(x^2+1)arctg(x)")
    print(f"Основной шаг h = {h}")
    print(f"Половинный шаг h/2 = {h_half}")
    print(f"Точность для метода секущих eps = {EPS}")

    # ------------------------- Метод стрельбы -------------------------
    eta, shooting_solution_full, iterations = shooting_method(h)
    eta_half, shooting_solution_half_full, _ = shooting_method(h_half)

    shooting_solution = [(x, y) for x, y, _ in shooting_solution_full]
    shooting_solution_half = [(x, y) for x, y, _ in
    ↪  shooting_solution_half_full]

    # Метод Рунге-Кутты 4-го порядка, значит p = 4.
    rr_shooting = runge_romberg_error(shooting_solution,
    ↪  shooting_solution_half, order=4)
```

```
    print_iterations(iterations)
    print(f"\nНайденный параметр стрельбы eta = y(0) = {eta:.10f}")
    print_solution_table("Метод стрельбы", shooting_solution, rr_shooting)

    # --------------------- Конечно-разностный метод ------------------
    fd_solution = finite_difference_method(h)
    fd_solution_half = finite_difference_method(h_half)

    # Из-за односторонней аппроксимации y'(0) по формуле (4.40)
    # вся конечно-разностная схема имеет первый порядок на границе, поэтому p =
    ↪  1.
    rr_fd = runge_romberg_error(fd_solution, fd_solution_half, order=1)

    print_solution_table("Конечно-разностный метод", fd_solution, rr_fd)


if __name__ == "__main__":
    main()
```

Вывод программы:

```
Лабораторная работа 4.2. Вариант 24
Краевая задача: (x^2 + 1)y'' - 2y = 0
Граничные условия: y'(0)=2, y(1)=3+pi/2
Точное решение: y=x^2+x+1+(x^2+1)arctg(x)
Основной шаг h = 0.1
Половинный шаг h/2 = 0.05
Точность для метода секущих eps = 1e-06

Итерации метода секущих в методе стрельбы
j        eta_j              y(b, eta_j)        Phi(eta_j)
-----------------------------------------------------------------------
0         0.0000000000       2.5707906030  -2.0000057238e+00
1         2.0000000000       6.5707839795   1.9999876527e+00
2         1.0000045178       4.5707963268  -8.8817841970e-16

Найденный параметр стрельбы eta = y(0) = 1.0000045178

Метод стрельбы
k      x_k          y_k              y_exact          |exact-y|        RR
-------------------------------------------------------------------------------
↪  -----------------
0     0.0000     1.0000045178     1.0000000000 4.5177864498e-06 2.8134566790e-07
1     0.1000     1.2106695671     1.2106653390 4.2281045165e-06 2.6332243994e-07
2     0.2000     1.4452953100     1.4452913822 3.9278041895e-06 2.4464758675e-07
3     0.3000     1.7076914976     1.7076879060 3.5916609145e-06 2.2374146851e-07
4     0.4000     2.0013906046     2.0013873975 3.2071249731e-06 1.9981615580e-07
5     0.5000     2.3295622836     2.3295595113 2.7723899869e-06 1.7275544257e-07
6     0.6000     2.6949728119     2.6949705204 2.2915749316e-06 1.4281412796e-07
7     0.7000     3.0999834572     3.0999816869 1.7702830579e-06 1.1034064065e-07
8     0.8000     3.5465763583     3.5465751452 1.2130598765e-06 7.5618177024e-08
9     0.9000     4.0363959568     4.0363953342 6.2260978062e-07 3.8815605308e-08
10    1.0000     4.5707963268     4.5707963268 8.8817841970e-16 0.0000000000e+00

Конечно-разностный метод
k      x_k          y_k              y_exact          |exact-y|        RR
-------------------------------------------------------------------------------
↪  -----------------
```

```
0     0.0000     1.0723465702     1.0000000000 7.2346570221e-02 3.8231814237e-02
1     0.1000     1.2723465702     1.2106653390 6.1681231205e-02 3.2575381539e-02
2     0.2000     1.4975415518     1.4452913822 5.2250169566e-02 2.7573415162e-02
3     0.3000     1.7515354094     1.7076879060 4.3847503414e-02 2.3118950408e-02
4     0.4000     2.0376675314     2.0013873975 3.6280133922e-02 1.9111166979e-02
5     0.5000     2.3589318522     2.3295595113 2.9372340915e-02 1.5457772431e-02
6     0.6000     2.7179390826     2.6949705204 2.2968562227e-02 1.2076580600e-02
7     0.7000     3.1169160054     3.0999816869 1.6934318475e-02 8.8961812110e-03
8     0.8000     3.5577307270     3.5465751452 1.1155581719e-02 5.8557840146e-03
9     0.9000     4.0419324086     4.0363953342 5.5370743669e-03 2.9044547826e-03
10    1.0000     4.5707963268     4.5707963268 0.0000000000e+00 0.0000000000e+00
```

4.2.5. Анализ результатов

В методе стрельбы за одну итерацию секущих найден параметр

$$\eta = y(0) = 1.0000045178.$$

Его максимальная ошибка на сетке составляет $4.52 \cdot 10^{-6}$. Максимальная ошибка конечно-разностного решения равна $7.23 \cdot 10^{-2}$, что связано с применением односторонней разности первого порядка в граничном условии.

4.3. Выводы

При решении задачи Коши метод Рунге--Кутты четвёртого порядка дал существенно меньшую погрешность, чем явный метод Эйлера и метод Адамса в реализованной схеме. В краевой задаче метод стрельбы на той же сетке оказался точнее конечно-разностного метода, поскольку последний теряет порядок аппроксимации из-за обработки производного граничного условия.

5. Итоги

В работе реализованы и проверены численные методы четырёх основных разделов: линейной алгебры, решения нелинейных уравнений, интерполяции и аппроксимации, решения обыкновенных дифференциальных уравнений. Для каждого задания представлены исходные данные варианта, расчётные формулы, полный текст программы и фактический вывод выполнения. Результаты расчётов подтверждают корректность реализованных алгоритмов и ожидаемые различия между методами по скорости сходимости и точности.